Unit-5

Unsupervised Learning – Clustering, pattern finding using association rules

- Unsupervised learning versus supervised learning,
- Applications of unsupervised learning,
- Clustering and its types,
- Apriori algorithm for association rule learning

Applications of unsupervised learning

- Segmentation of target consumer populations by **an advertisement** consulting agency on the basis of few dimensions such as demography, financial data, purchasing habits, etc. so that the advertisers can reach their Larget consumers efficiently
- Anomaly **or fraud detection** in the banking sector by identifying the pattern of loan defaulters
- **Image processing** and image segmentation such as face recognition, expression identification, etc.
- Grouping of important **characteristics in genes** to identify important influencers in new areas of genetics.
- Utilization by data scientists **to reduce the dimensionalities** in sample data to simplify modelling
- **Document clustering** and identifying potential labelling options

Clustering

- **Clustering** or **cluster analysis** is a machine learning technique, **which groups the unlabelled dataset.**

- It can be defined as "***A way of grouping the data points into different clusters, consisting of similar data points.*** ***The objects with the possible similarities remain in a group that has less or no similarities with another group.***"

- It does it by **finding some similar patterns in the unlabelled dataset such as shape, size, color, behavior, etc**., and **divides them as per the presence and absence of those similar patterns.**

- **It is an unsupervised learning method**, hence no supervision is provided to the algorithm, and it **deals with the unlabeled dataset**.

- After applying this clustering technique**, each cluster or group is provided with a cluster-ID.**

- **ML system can use this id to simplify the processing of large and complex datasets.**

- The clustering technique is commonly **used for statistical data analysis.**

- Clustering is somewhere similar to the classification algorithm, but the difference is the type of dataset that we are using.

- In classification, we work with the labeled data set, whereas in clustering, we work with the unlabelled dataset.

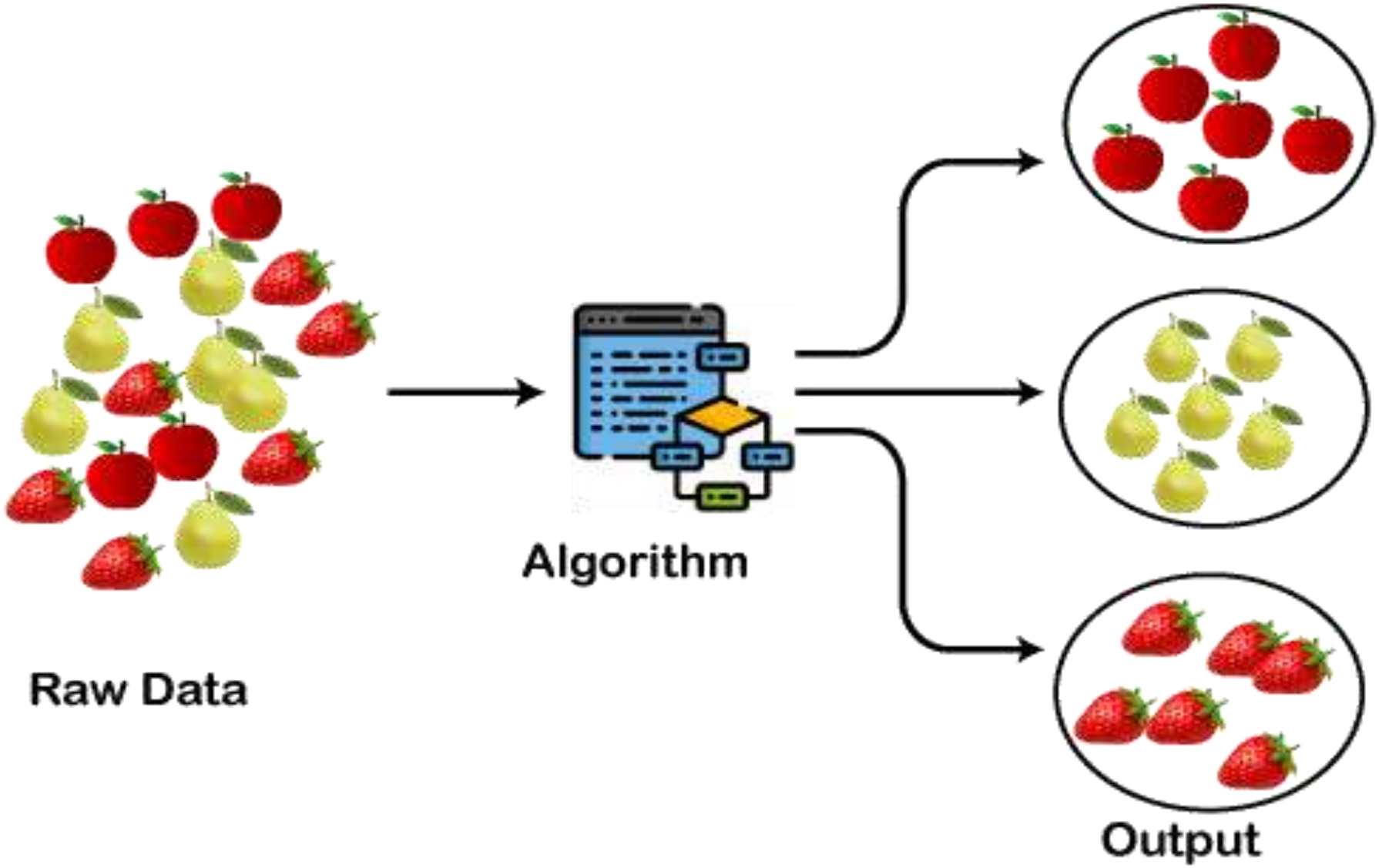
Example

- **Clustering technique with the real-world example of Mall:** When we visit any shopping mall, we can observe that the things with similar usage are grouped together. Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things. The clustering technique also works in the same way. Other examples of clustering are grouping documents according to the topic.

- Apart from these general usages, it is used by the **Amazon** in its recommendation system to provide the recommendations as per the past search of products.

- **Netflix** also uses this technique to recommend the movies and web-series to its users as per the watch history.

We can see the different fruits are divided into several groups with similar properties.



Types of Clustering Methods

- The clustering methods are broadly divided into **Hard clustering** (datapoint belongs to only one group) and **Soft Clustering** (data points can belong to another group also). But there are also other various approaches of Clustering exist.

- Below are the main clustering methods used in Machine learning:
  - **Partitioning Clustering**
  - **Density-Based Clustering**
  - **Distribution Model-Based Clustering**
  - **Hierarchical Clustering**
  - **Fuzzy Clustering**

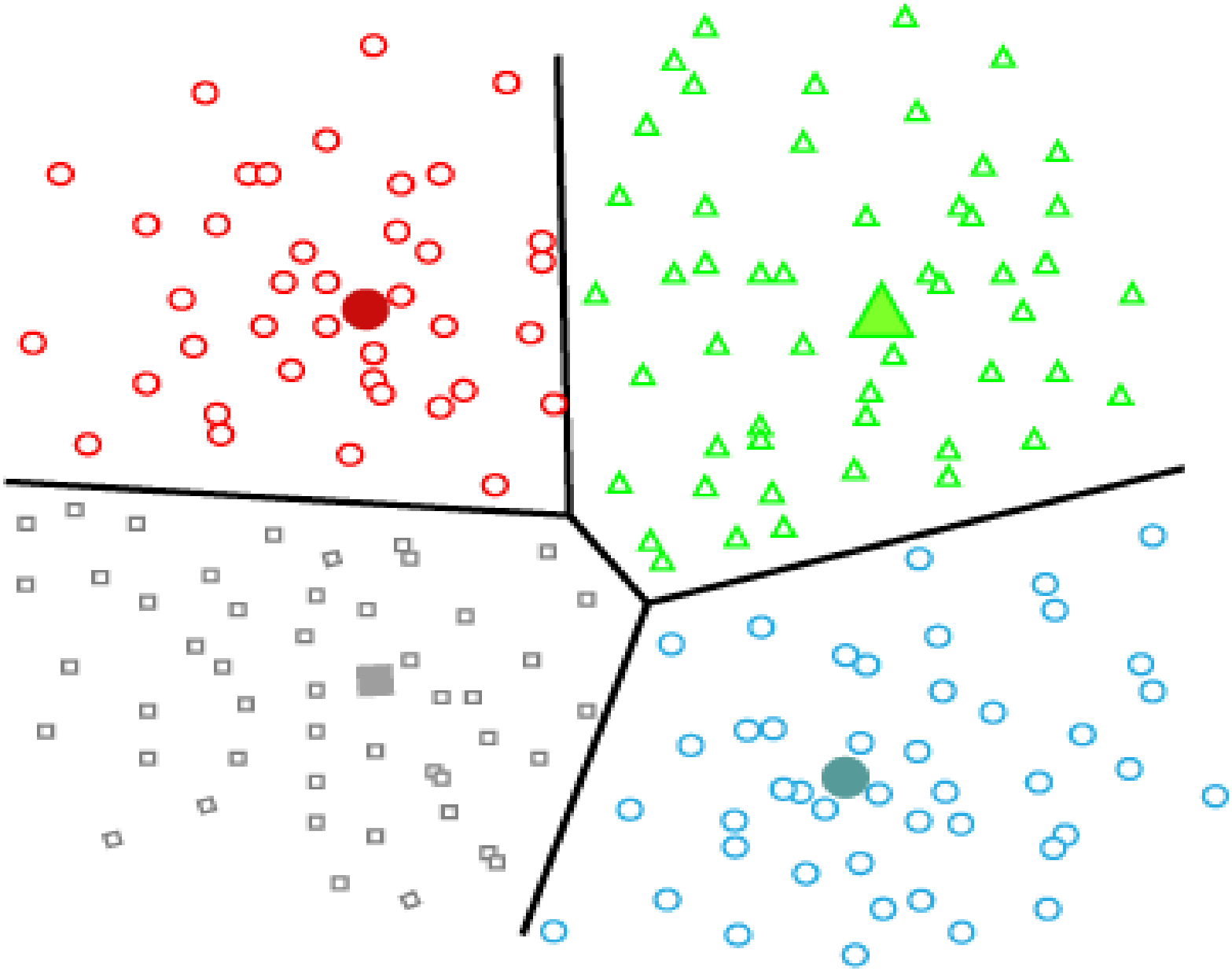
Partitioning Clustering

- It is a type of clustering that **divides the data into non-hierarchical groups**. It is also known as the **centroid-based method**.

- The most common example of partitioning clustering is the **K-Means Clustering algorithm**.

- In this type, the **dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups.**

- The cluster center is created in such a way that the distance between the data points of one cluster is minimum as compared to another cluster centroid.



K-Means Clustering Algorithm

- K-Means Clustering is an Unsupervised Learning algorithm, which groups the unlabeled dataset into different clusters.

- Here K defines the number of pre-defined clusters that need to be created in the process, as if K=2, there will be two clusters, and for K=3, there will be three clusters, and so on.

- It is an iterative algorithm that divides **the unlabeled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.**

- It allows us to cluster the data into different groups and a convenient way to **discover the categories of groups in the unlabeled dataset on its own without the need for any training.**

- It is a **centroid-based algorithm**, where **each cluster is associated with a centroid.**

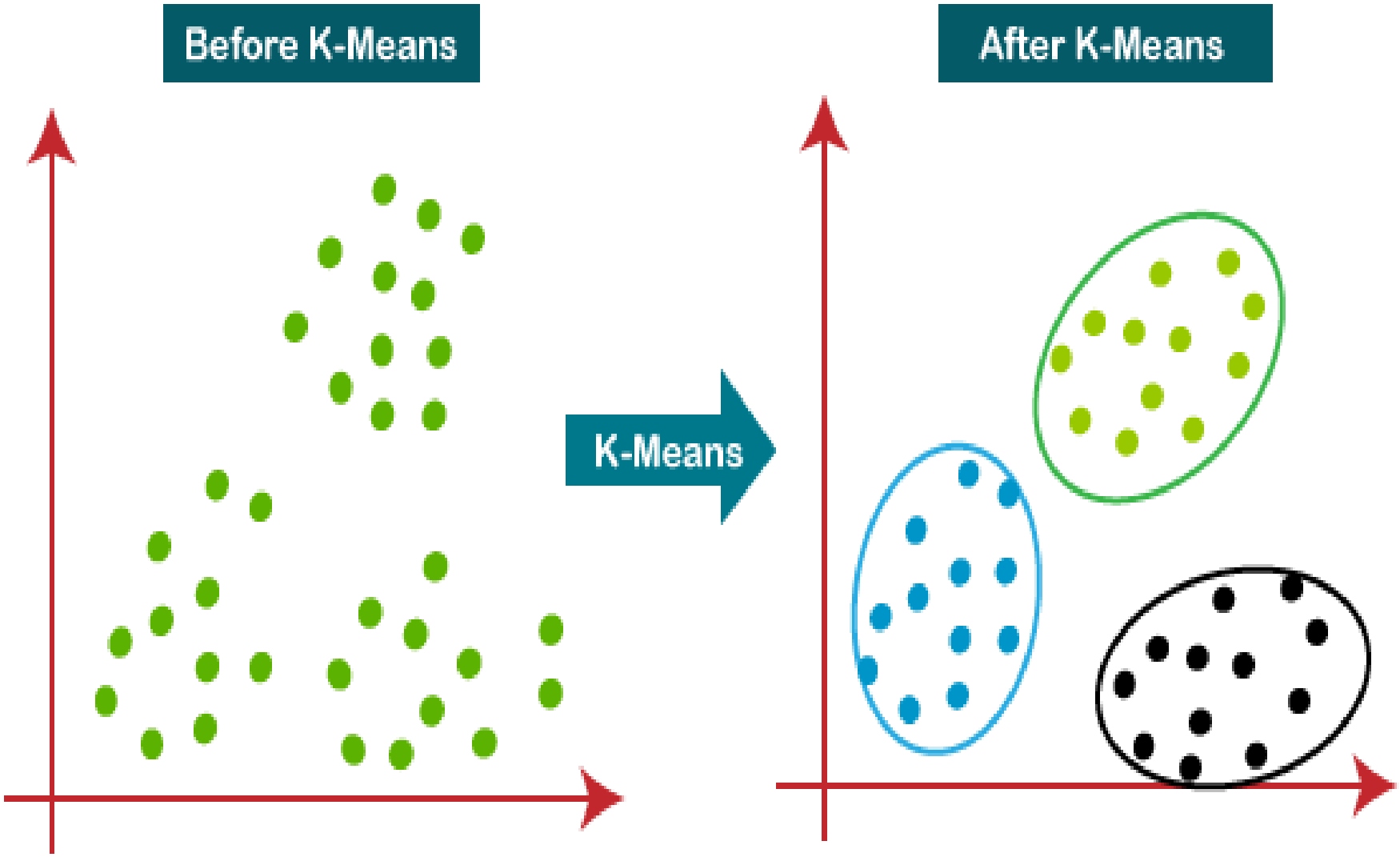
- The **main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters**.

- The algorithm takes the **unlabeled dataset as input**, **divides the dataset into k-number of clusters**, and **repeats the process until it does not find the best clusters**.

- **The value of k should be predetermined in this algorithm.**

- The k-means clustering algorithm mainly performs two tasks:
  - **Determines the best value for K center points or centroids by an iterative process.**
  - **Assigns each data point to its closest k-center**. Those data points which are near to the particular k-center, create a cluster.

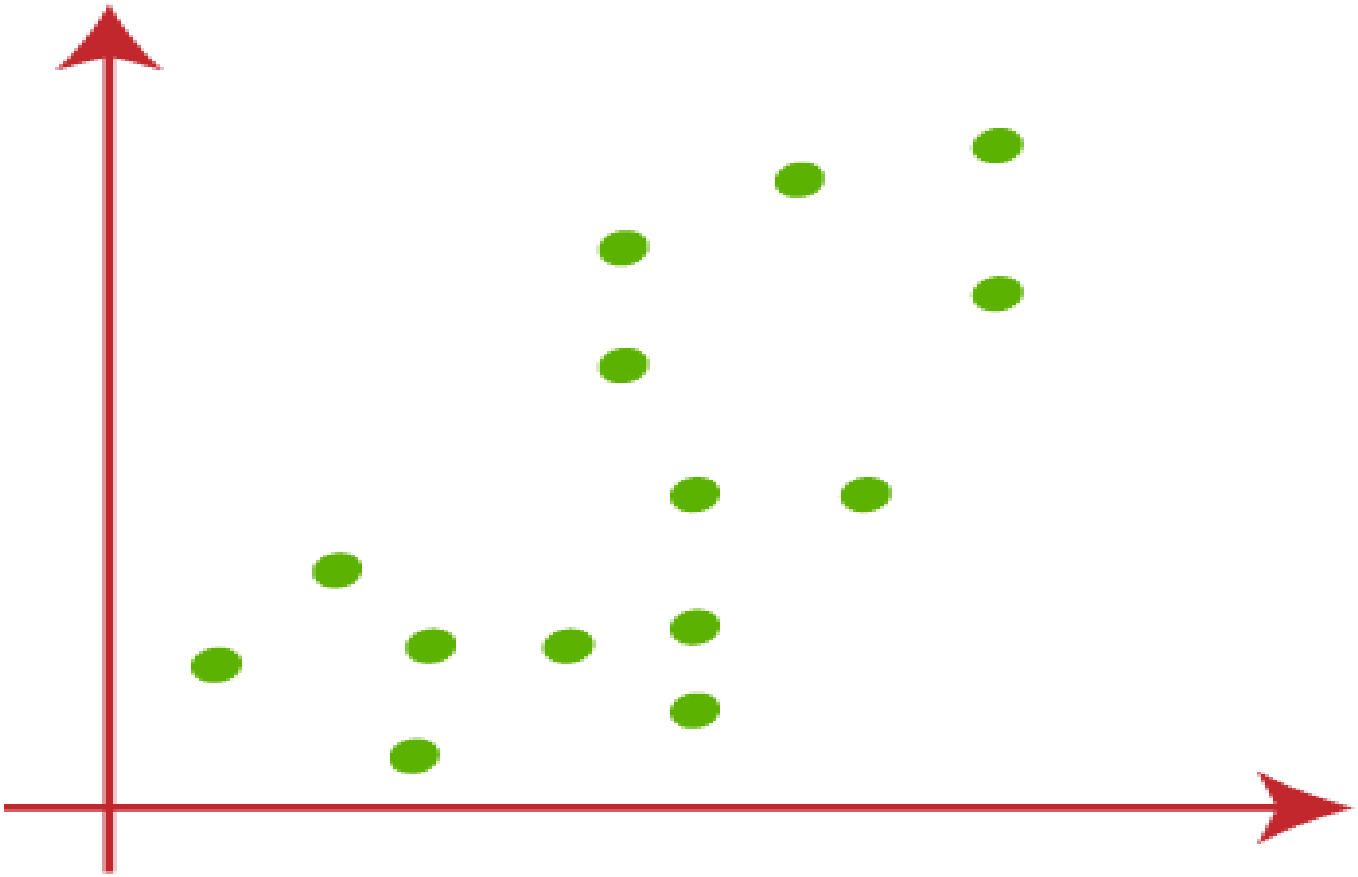
- Hence **each cluster has datapoints with some commonalities, and it is away from other clusters.**


Before K-Means
K-Means
After K-Means

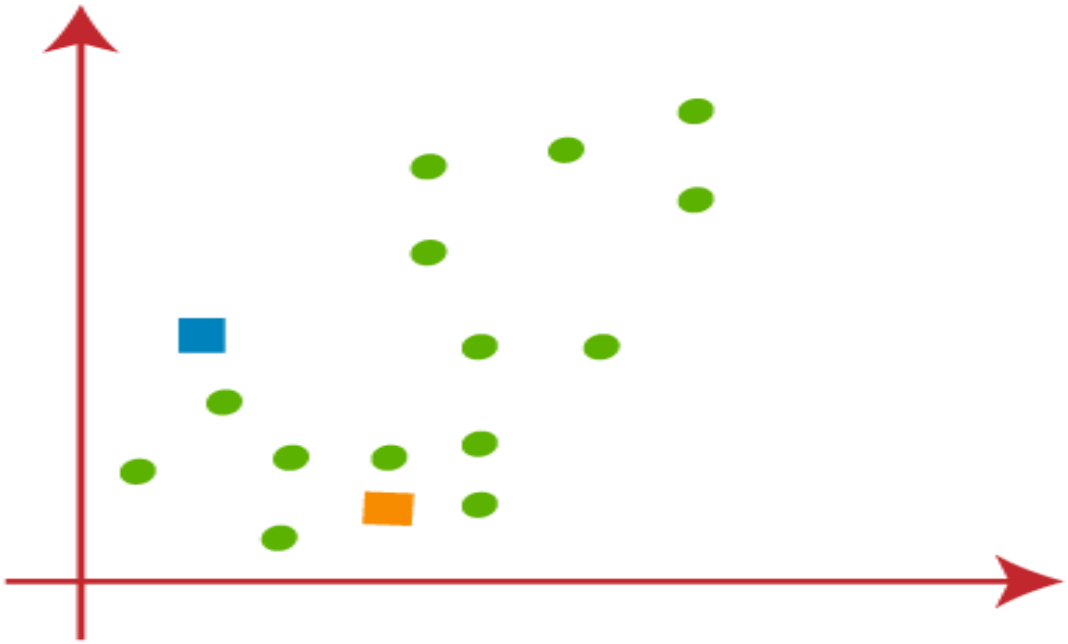
How does the K-Means Algorithm Work?

- **Step-1:** Select the number K to decide the number of clusters.
- **Step-2:** Select random K points or centroids. (It can be other from the input dataset).
- **Step-3:** Assign each data point to their closest centroid, which will form the predefined K clusters.
- **Step-4:** Calculate the variance and place a new centroid of each cluster.
- **Step-5:** Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.
- **Step-6:** If any reassignment occurs, then go to step-4 else go to FINISH.
- **Step-7**: The model is ready.

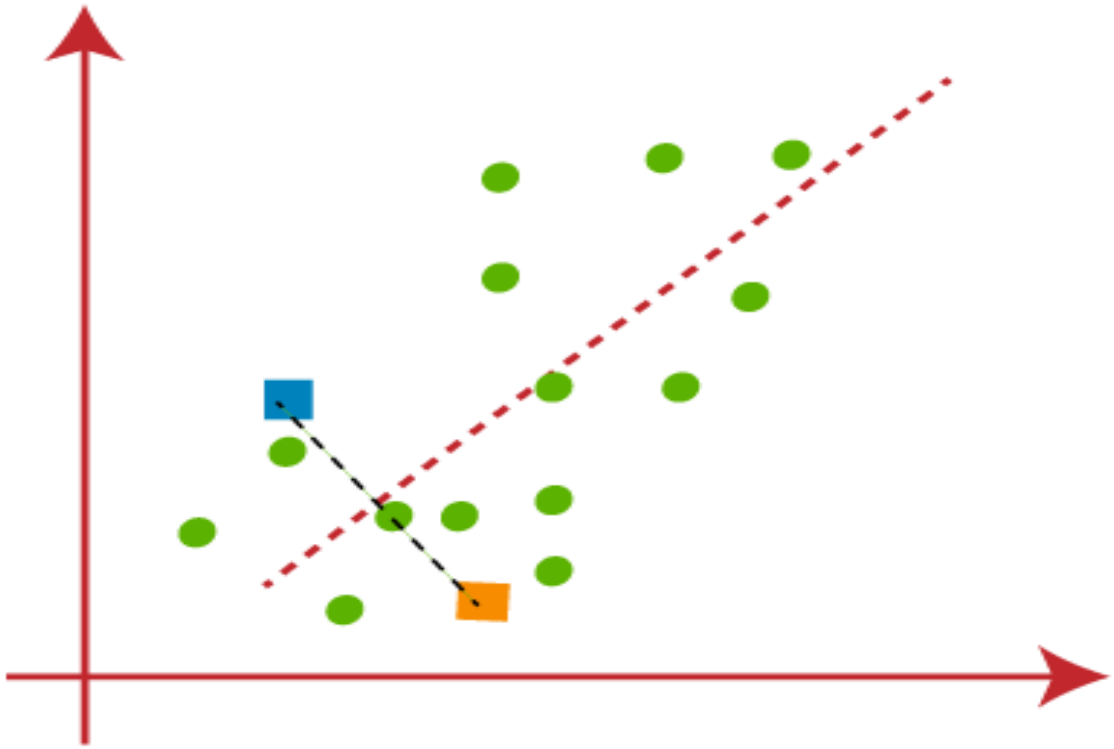
Suppose we have two variables M1 and M2. The x-y axis scatter plot of these two variables is given below:



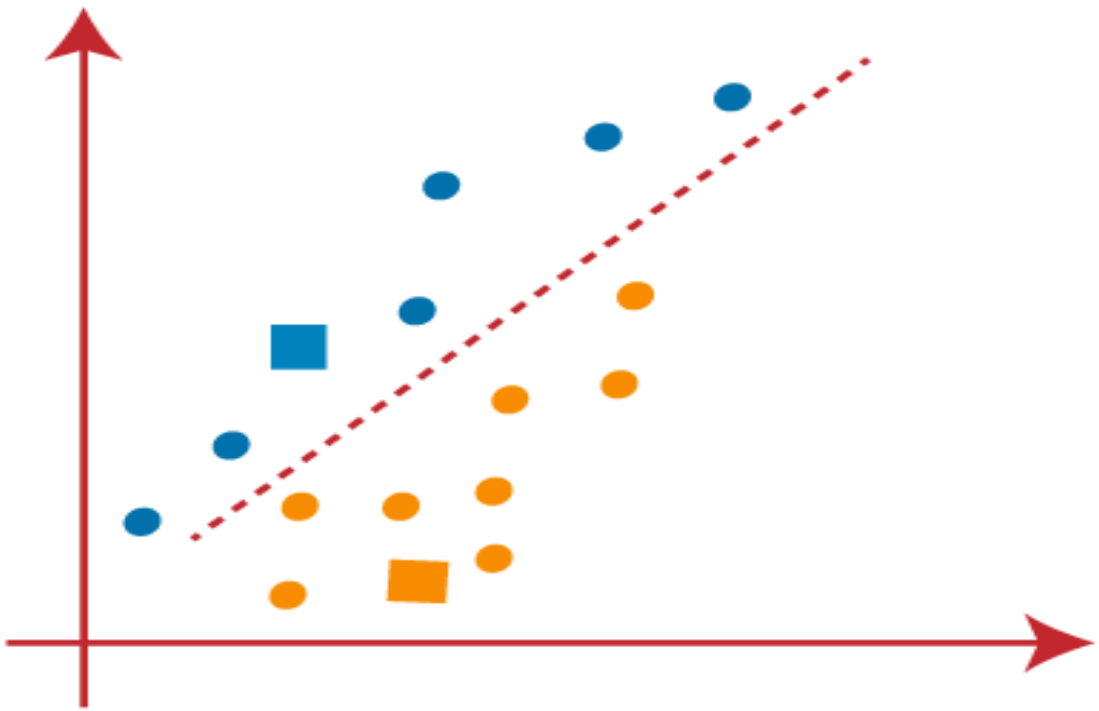
- Let's take number k of clusters, i.e., K=2, to identify the dataset and to put them into different clusters. It means here we will try to group these datasets into two different clusters.
- We need to choose some random k points or centroid to form the cluster. These points can be either the points from the dataset or any other point. So, here we are selecting the below two points as k points, which are not the part of our dataset.



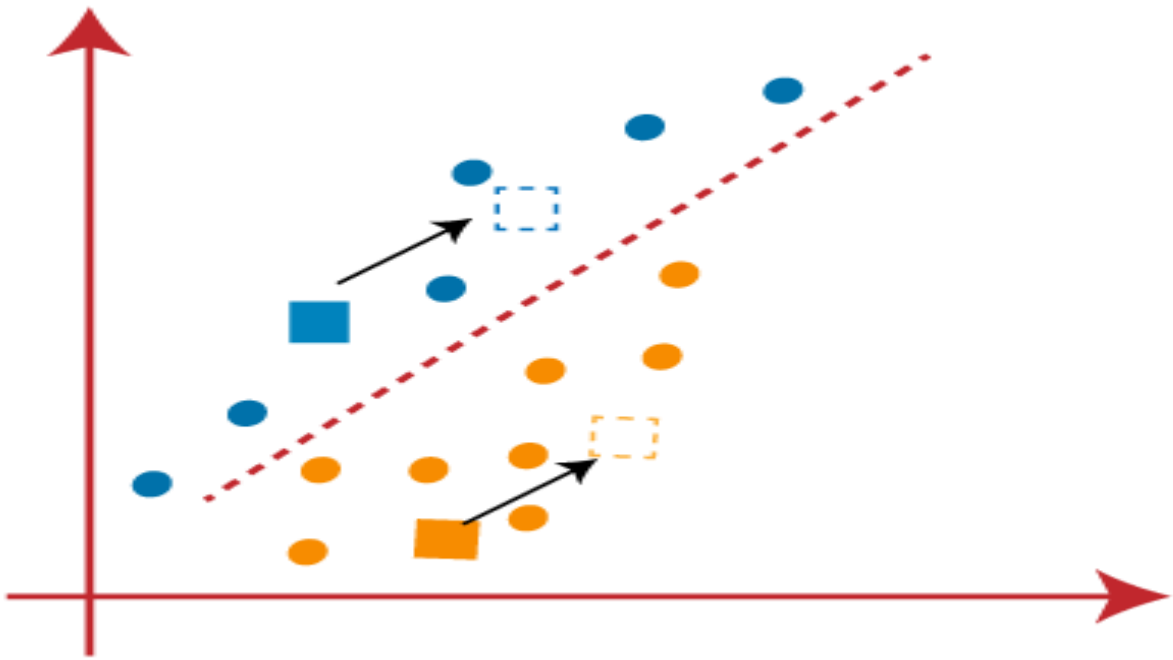
- Now we will assign each data point of the scatter plot to its closest K-point or centroid. We will compute it by applying some mathematics that we have studied to calculate the distance between two points. So, we will draw a median between both the centroids.



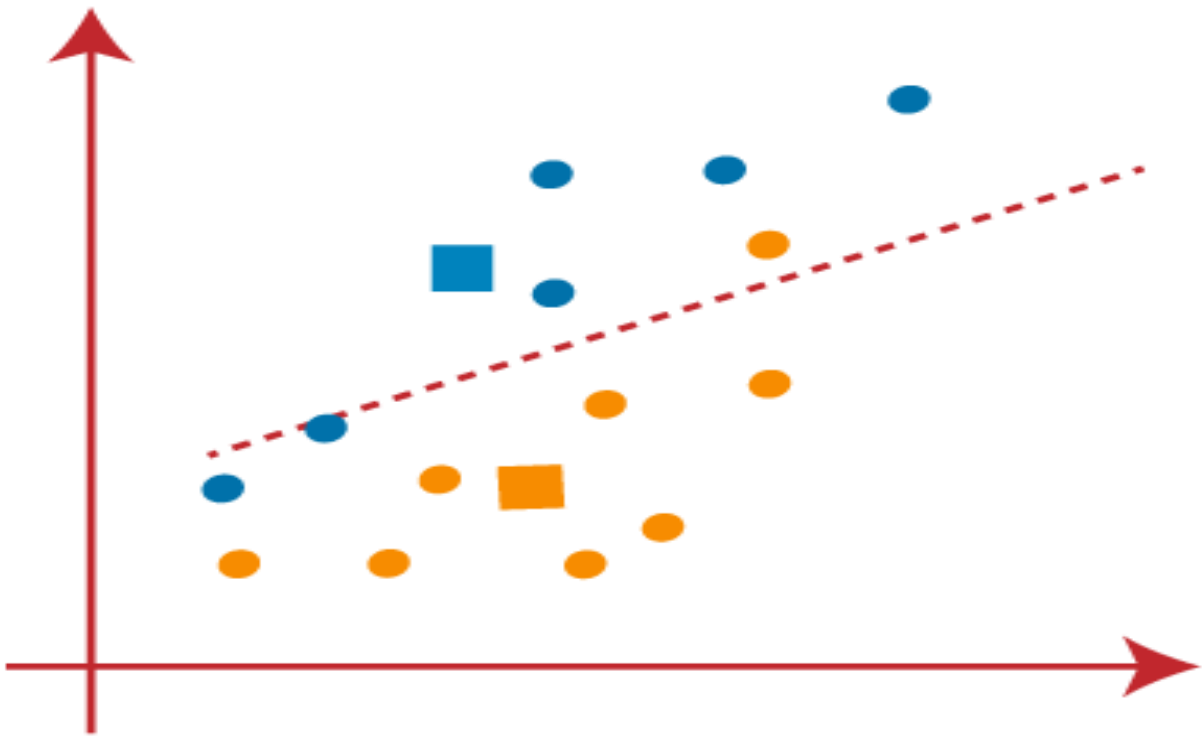
- From the above image, it is clear that points left side of the line is near to the K1 or blue centroid, and points to the right of the line are close to the yellow centroid. Let's color them as blue and yellow for clear visualization.



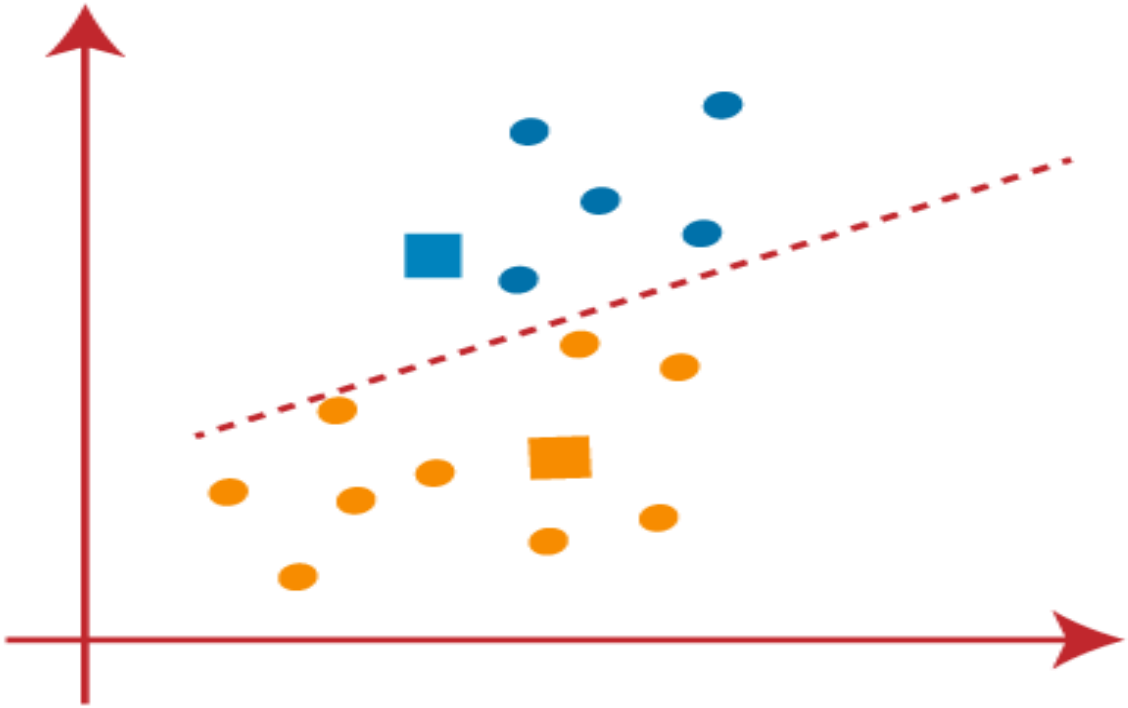
- As we need to find the closest cluster, so we will repeat the process by choosing **a new centroid**. To choose the new centroids, we will compute the center of gravity of these centroids, and will find new centroids as below:



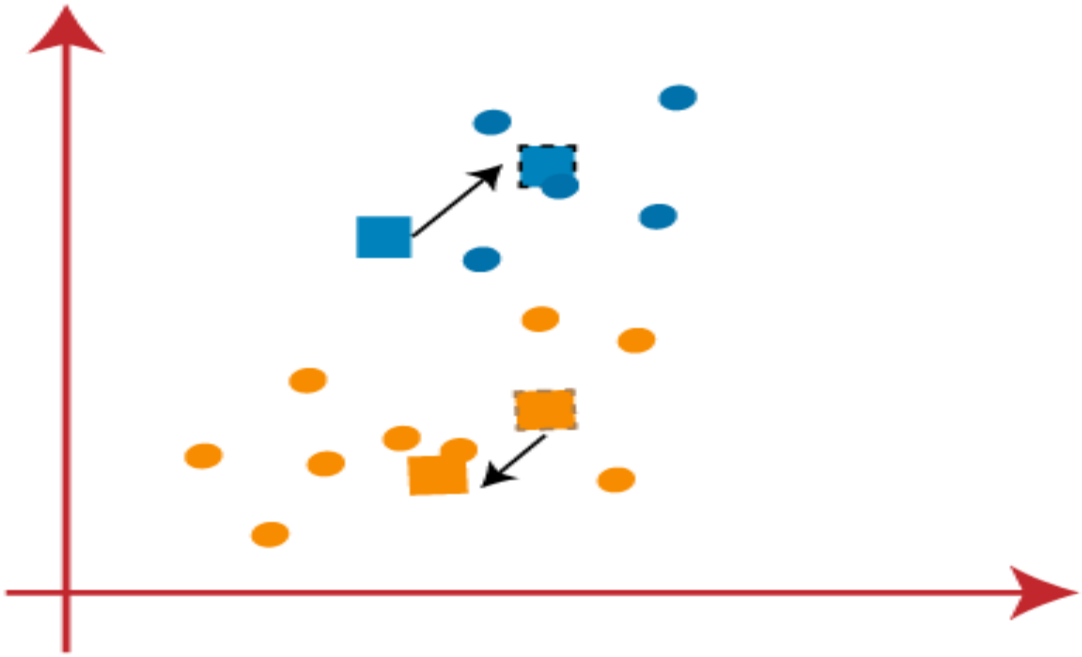
- Next, we will reassign each datapoint to the new centroid. For this, we will repeat the same process of finding a median line. The median will be like below image:



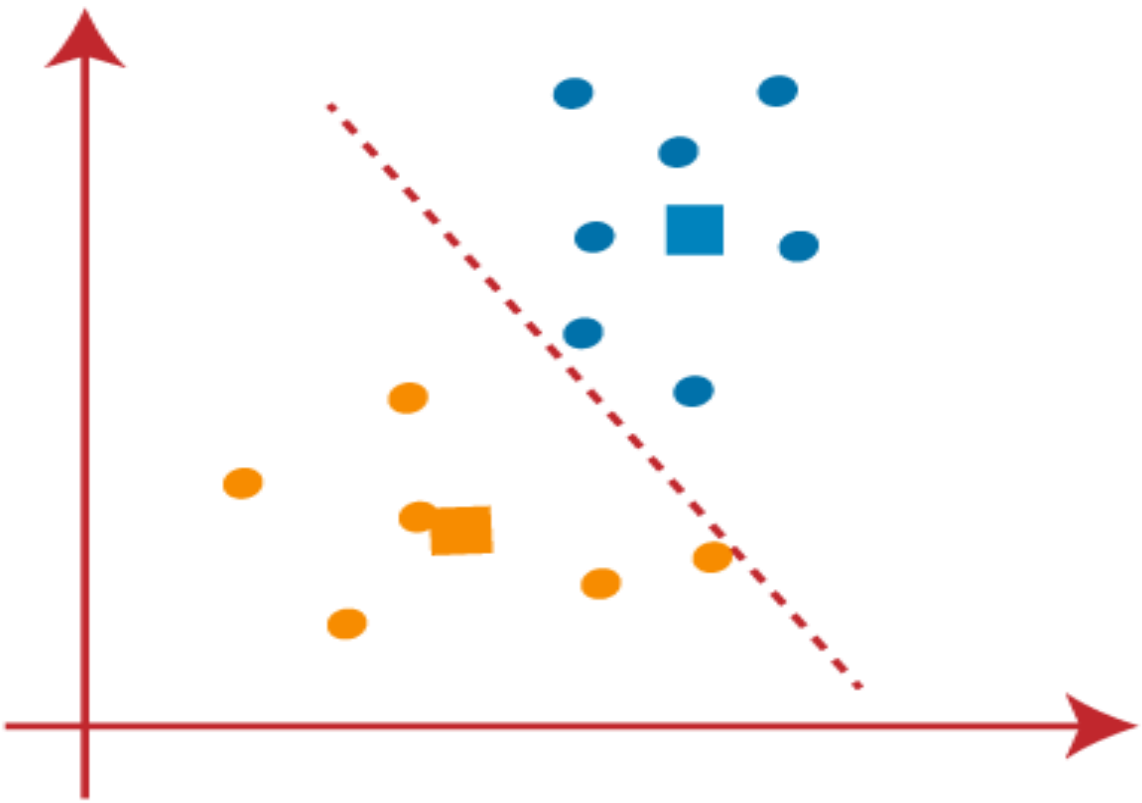
- From the above image, we can see, one yellow point is on the left side of the line, and two blue points are right to the line. So, these three points will be assigned to new centroids.



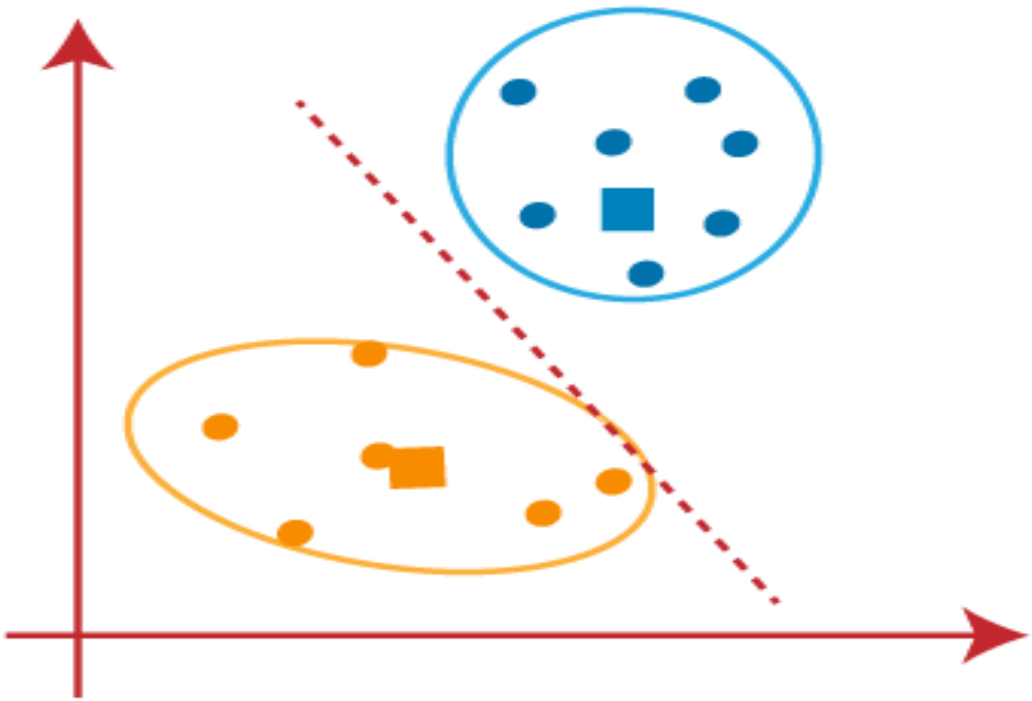
- As reassignment has taken place, so we will again go to the step-4, which is finding new centroids or K-points.
- We will repeat the process by finding the center of gravity of centroids, so the new centroids will be as shown in the below image:



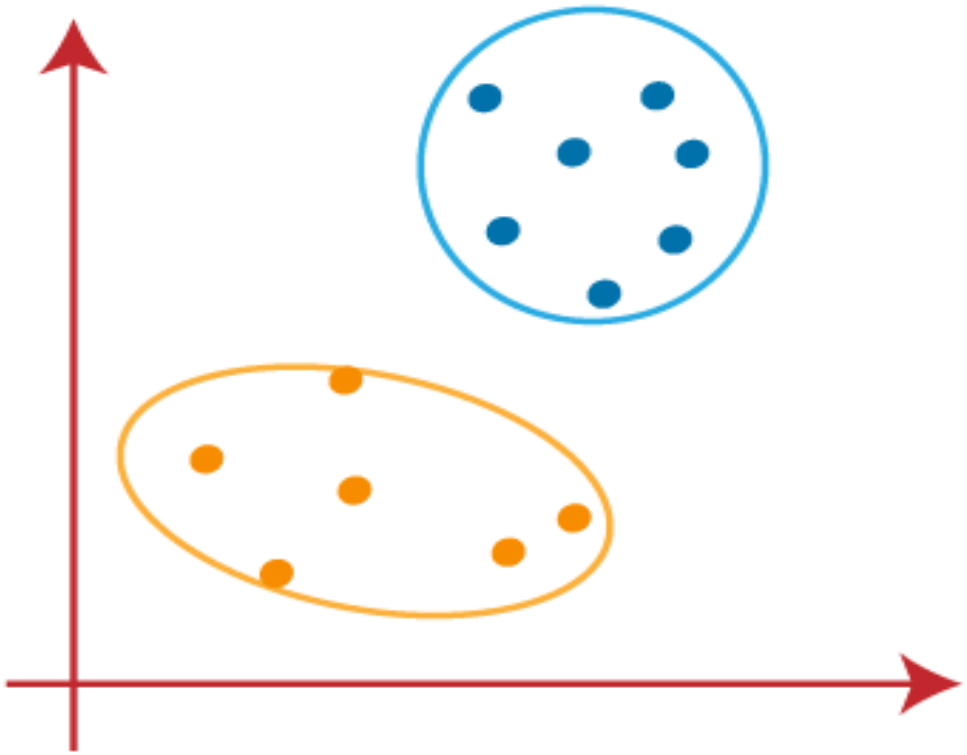
- As we got the new centroids so again will draw the median line and reassign the data points. So, the image will be:



- We can see in the above image; there are no dissimilar data points on either side of the line, which means our model is formed. Consider the below image:



- As our model is ready, so we can now remove the assumed centroids, and the two final clusters will be as shown in the below image:



How to choose the value of "K number of clusters" in K-means Clustering?

- The performance of the K-means clustering algorithm depends upon highly efficient clusters that it forms. But choosing the optimal number of clusters is a big task.

- There are some different ways to find the optimal number of clusters,
- **Elbow Method**
- **Purpose Based**

1. Elbow Method

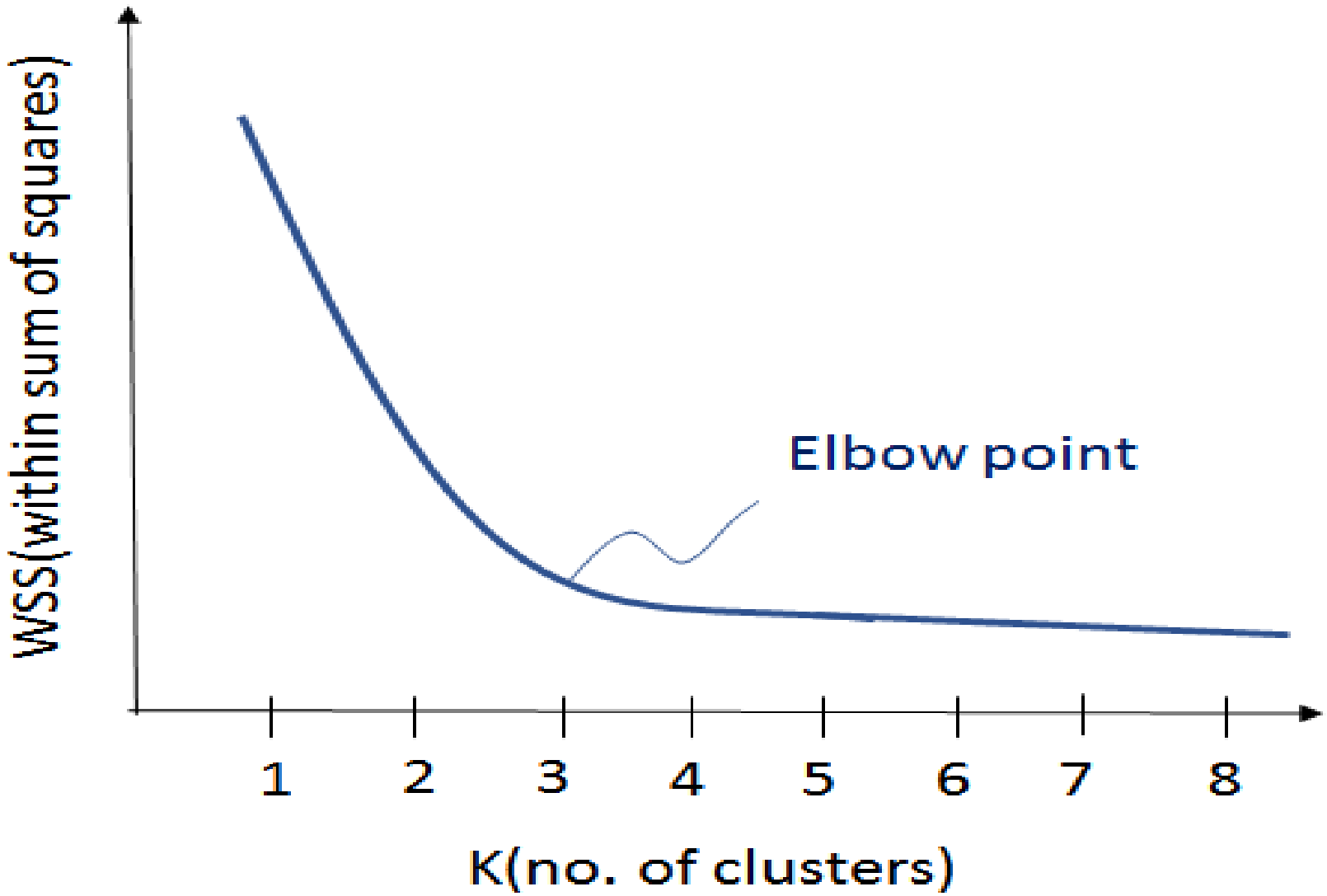
- The Elbow method is one of the most popular ways to find the optimal number of clusters.
- This method uses the concept of WCSS value.
- **WCSS** stands for **Within Cluster Sum of Squares**, which defines the total variations within a cluster.
- It is called elbow method because the curve looks like a human arm and the elbow point gives us the optimum number of clusters.
  - **WCSS=** $\sum_{\text{Pi in Cluster1}} \textbf{distance}(P_i\ C_1)^2 + \sum_{\text{Pi in Cluster2}} \textbf{distance}(P_i\ C_2)^2 + \sum_{\text{Pi in CLuster3}} \textbf{distance}(P_i\ C_3)^2$
- $\sum_{\text{Pi in Cluster1}} \textbf{distance}(P_i\ C_1)^2$: It is the sum of the square of the distances between each data point and its centroid within a cluster1 and the same for the other two terms.

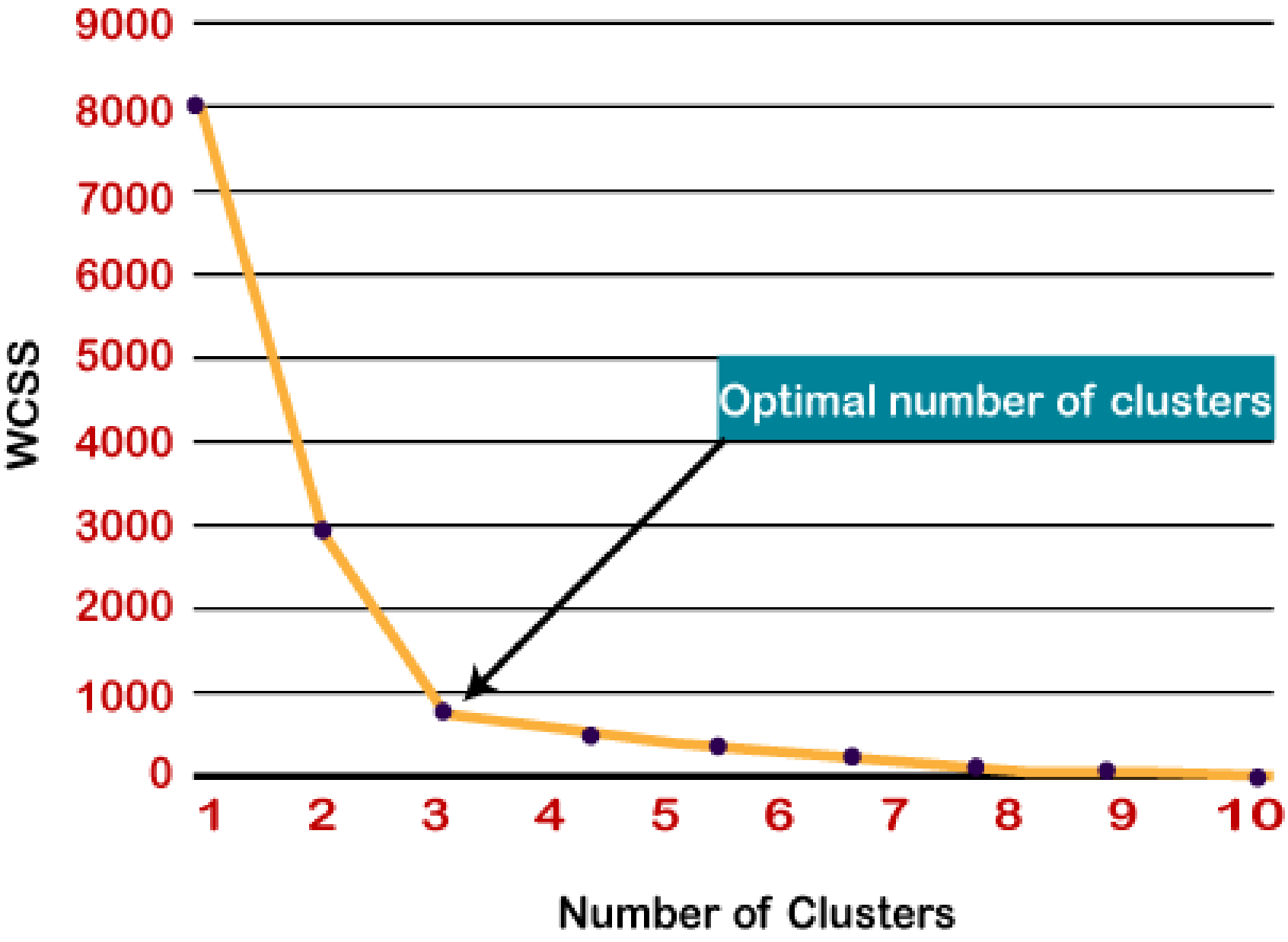
- To measure the **distance between data points and centroid, we can use any method such as Euclidean distance or Manhattan distance.**

- To find the optimal value of clusters, the elbow method follows the below steps:
  - It executes the K-means clustering on a given dataset for **different K values (ranges from 1-10).**
  - For **each value of K, calculates the WCSS value**.
  - Plots a **curve between calculated WCSS values and the number of clusters K.**
  - The sharp point **of bend or a point of the plot looks like an arm**, then that **point is considered as the best value of K**.

- **Note:** We can choose the number of clusters equal to the given data points. If **we choose the number of clusters equal to the data points, then the value of WCSS becomes zero, and that will be the endpoint of the plot**.

Elbow Method




9000
8000
7000
6000
5000
4000
3000
2000
1000
0
WCSS
Optimal number of clusters
1
2
3
4
5
6
7
8
9
10
Number of Clusters

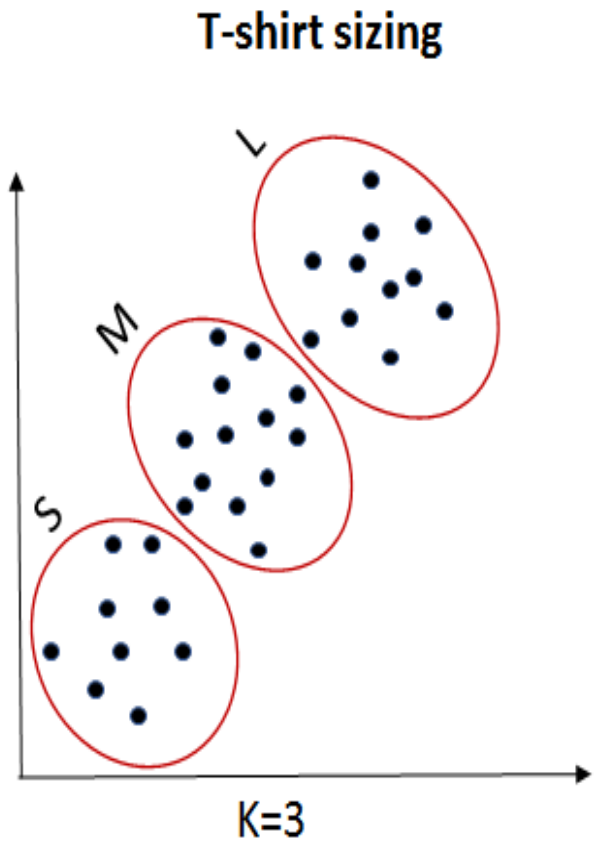
2. Purpose Based

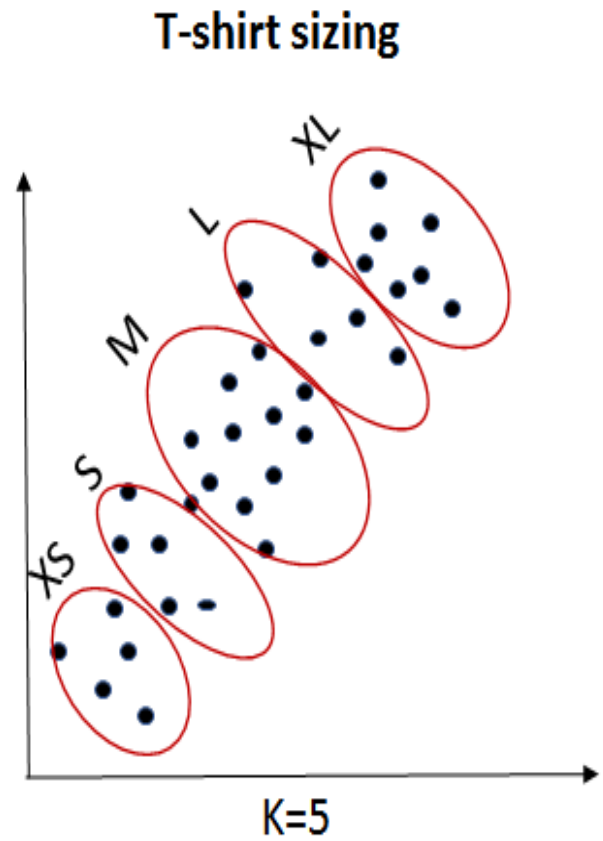
- You can run **k-means clustering algorithm to get different clusters based on a variety of purposes.**

- You can **partition the data on different metrics and see how well it performs for that particular case.**

- Let's take an example of marketing T-shirts of different sizes. You can partition the dataset into different number of clusters depending upon the purpose that you want to meet.

- In the following example, have taken two different criteria, price and comfort.





- **K=3:** If you want to provide only **3 sizes(S, M, L)** so that prices are cheaper, you will divide the data set into 3 clusters.
- **K=5:** Now, if you want to provide more comfort and variety to your customers with more **sizes (XS, S, M, L, XL)**, then you will divide the data set into 5 clusters.

Hierarchical clustering

- Hierarchical clustering is another unsupervised machine learning algorithm, which is **used to group the unlabeled datasets into a cluster** and also known as **hierarchical cluster analysis** or HCA.

- In this algorithm, we develop the **hierarchy of clusters in the form of a tree**, and this **tree-shaped structure is known as the dendrogram.**

- Sometimes the results of K-means clustering and hierarchical clustering may look similar, but they both differ depending on how they work.

- As there **is no requirement to predetermine the number of clusters as we did in the K-Means algorithm.**

- The hierarchical clustering technique has two approaches:
  - **Agglomerative:** Agglomerative is a **bottom-up** approach, in which the algorithm **starts with taking all data points as single clusters** and **merging them until one cluster is left**.
  - **Divisive:** Divisive algorithm is the reverse of the agglomerative algorithm as it is a **top-down approach.**

Agglomerative Hierarchical clustering

- The agglomerative hierarchical clustering algorithm is a popular example of HCA. To group the datasets into clusters, it follows the **bottom-up approach**.

- It means, this algorithm considers **each dataset as a single cluster at the beginning, and then start combining the closest pair of clusters together.**

- It does this **until all the clusters are merged into a single cluster that contains all the datasets.**

- This hierarchy of clusters is **represented in the form of the dendrogram.**

How the Agglomerative Hierarchical clustering Work?

1. Make **each data point a single-point cluster** → forms N clusters.

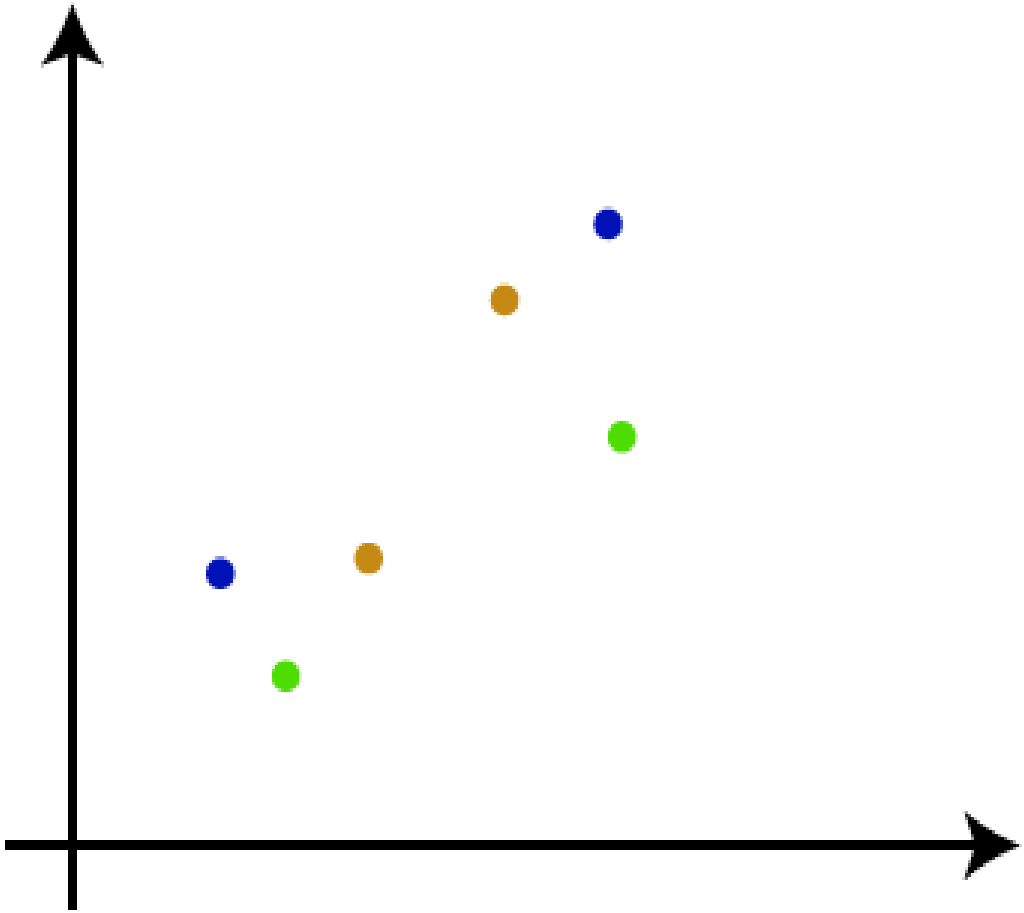
2. **Take the two closest data points and make them one cluster** → forms N-1 clusters.

3. **Take the two closest clusters and make them one cluster** → Forms N-2 clusters.

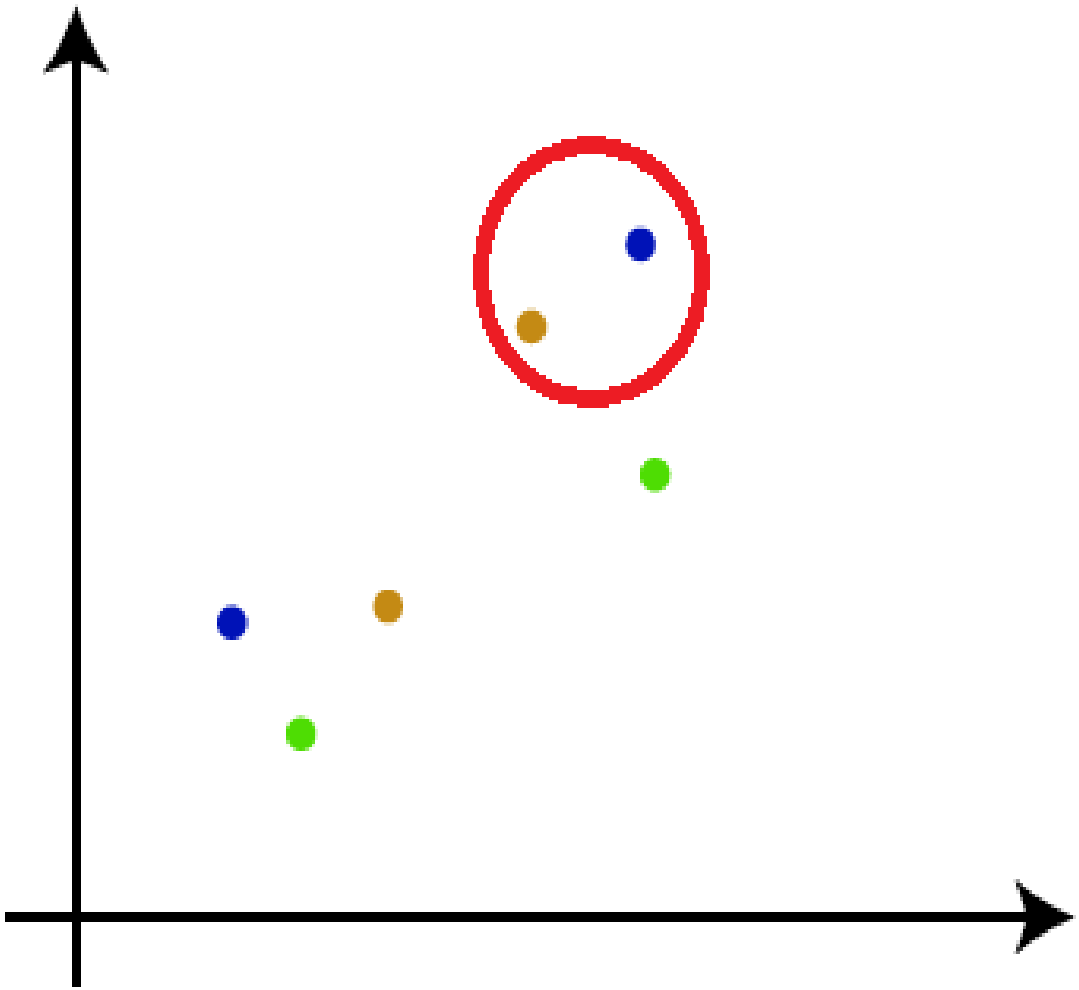
4. **Repeat step-3 until you are left with only one** cluster.

5. Once all the clusters are combined into one big cluster, develop the dendrogram to divide the clusters as per the problem.

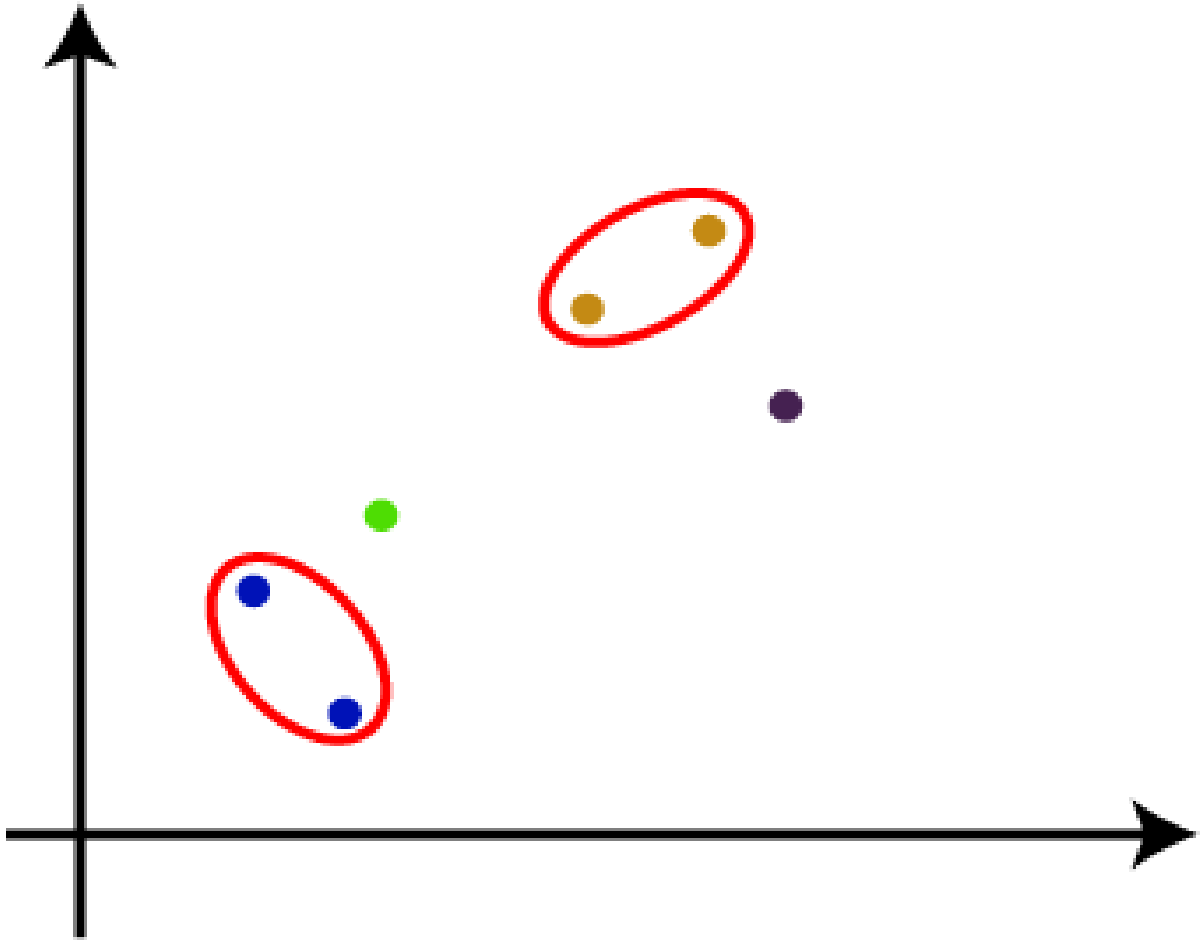
- **Step-1:** Create each data point as a single cluster. Let's say there are N data points, so the number of clusters will also be N.



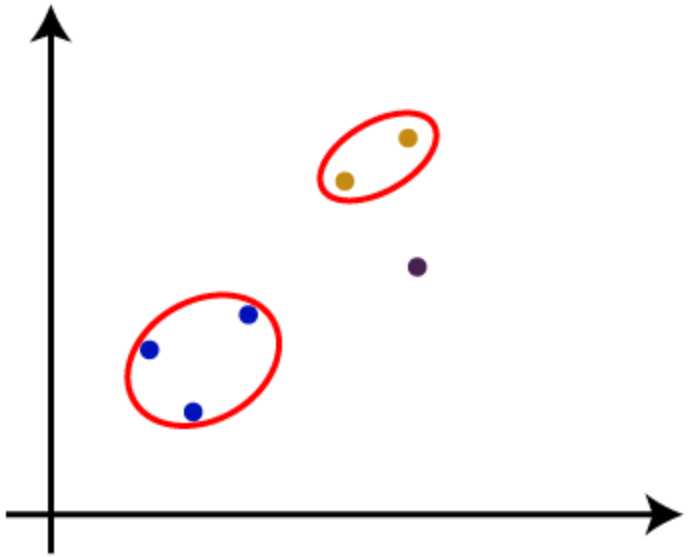
- **Step-2:** Take two closest data points or clusters and merge them to form one cluster. So, there will now be N-1 clusters.

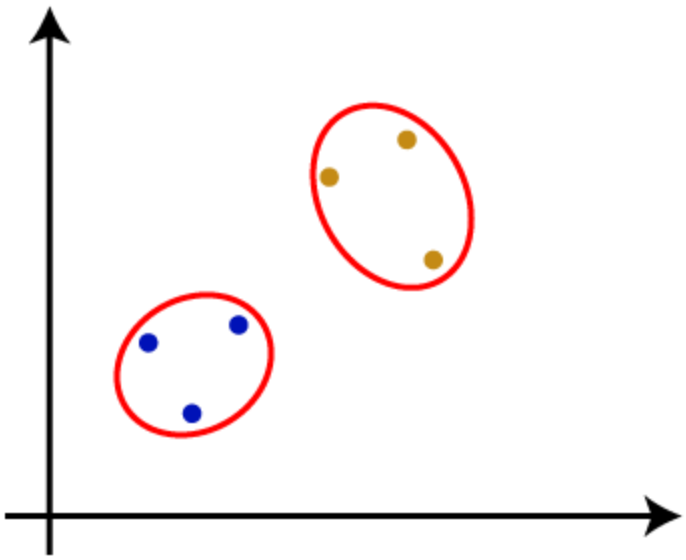


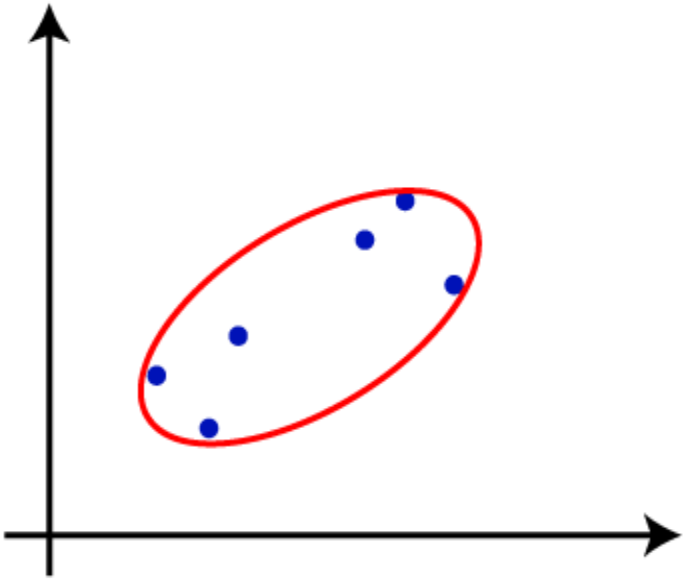
- **Step-3**: Again, take the two closest clusters and merge them together to form one cluster. There will be N-2 clusters.

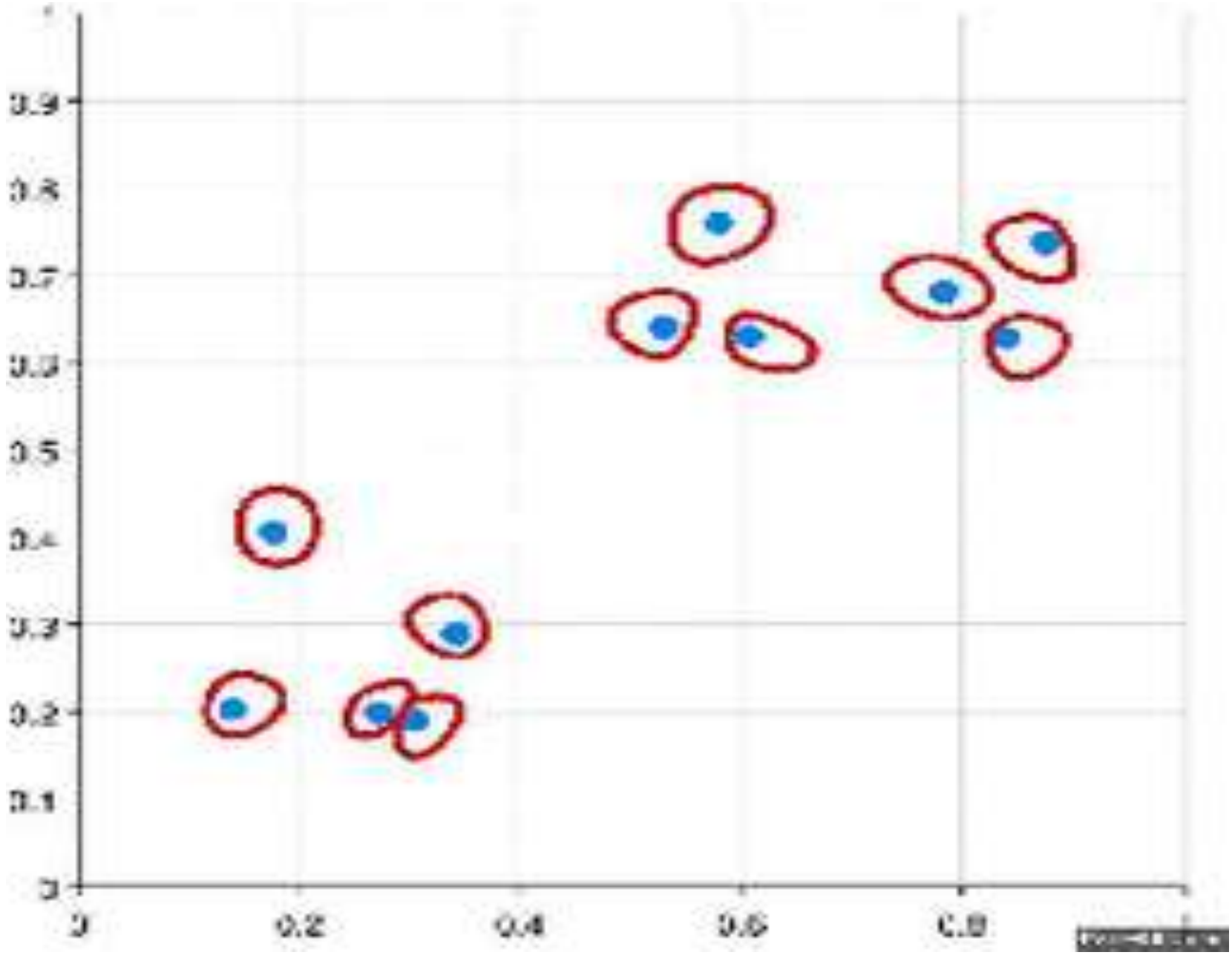


- **Step-4:** Repeat Step 3 until only one cluster left. So, we will get the following clusters.








0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.2
0.4
0.6
0.8

Measure for the distance between two clusters

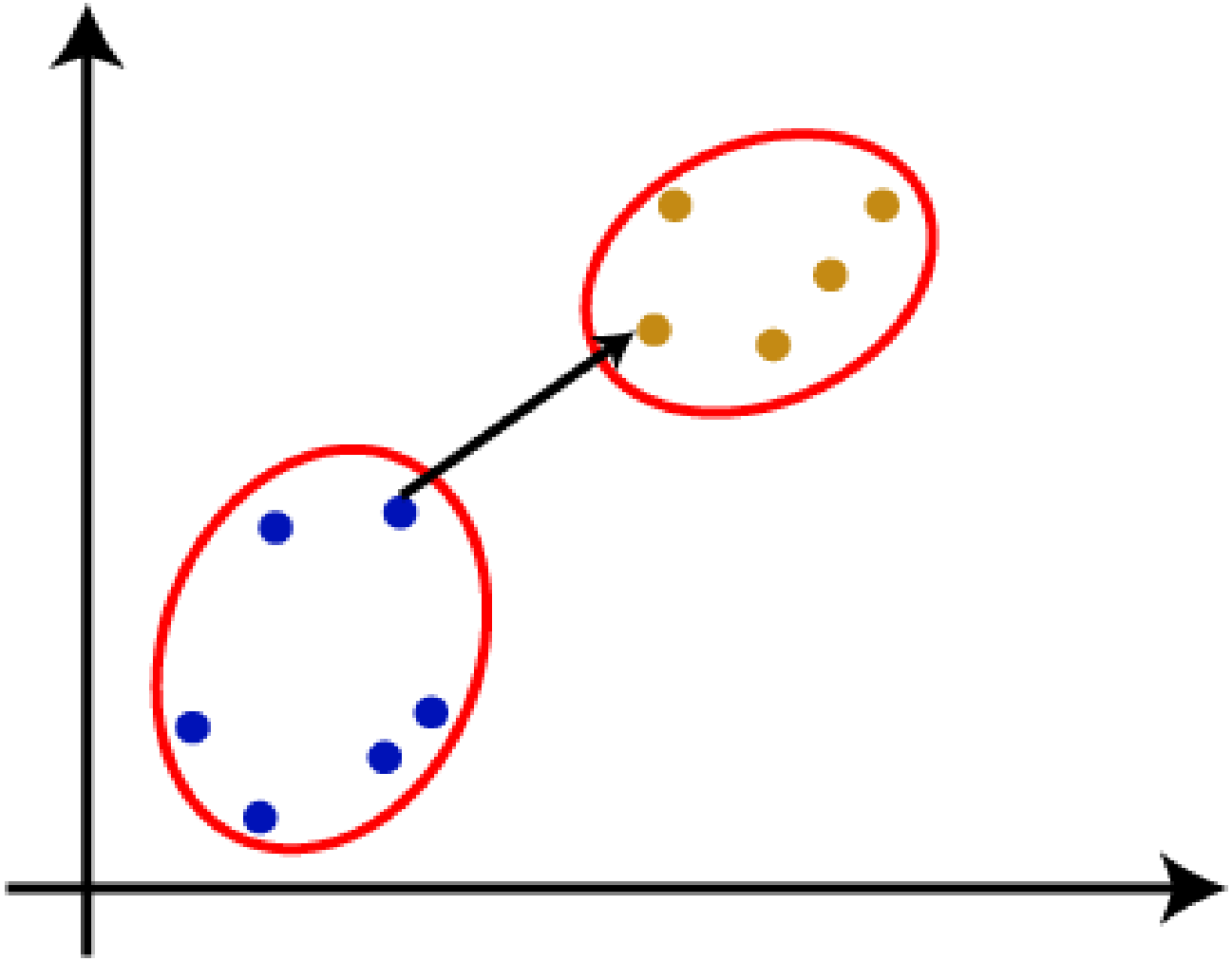
- the **closest distance** between the two clusters is crucial for the hierarchical clustering.

- There are **various ways to calculate the distance between two clusters, and these ways decide the rule for clustering**.

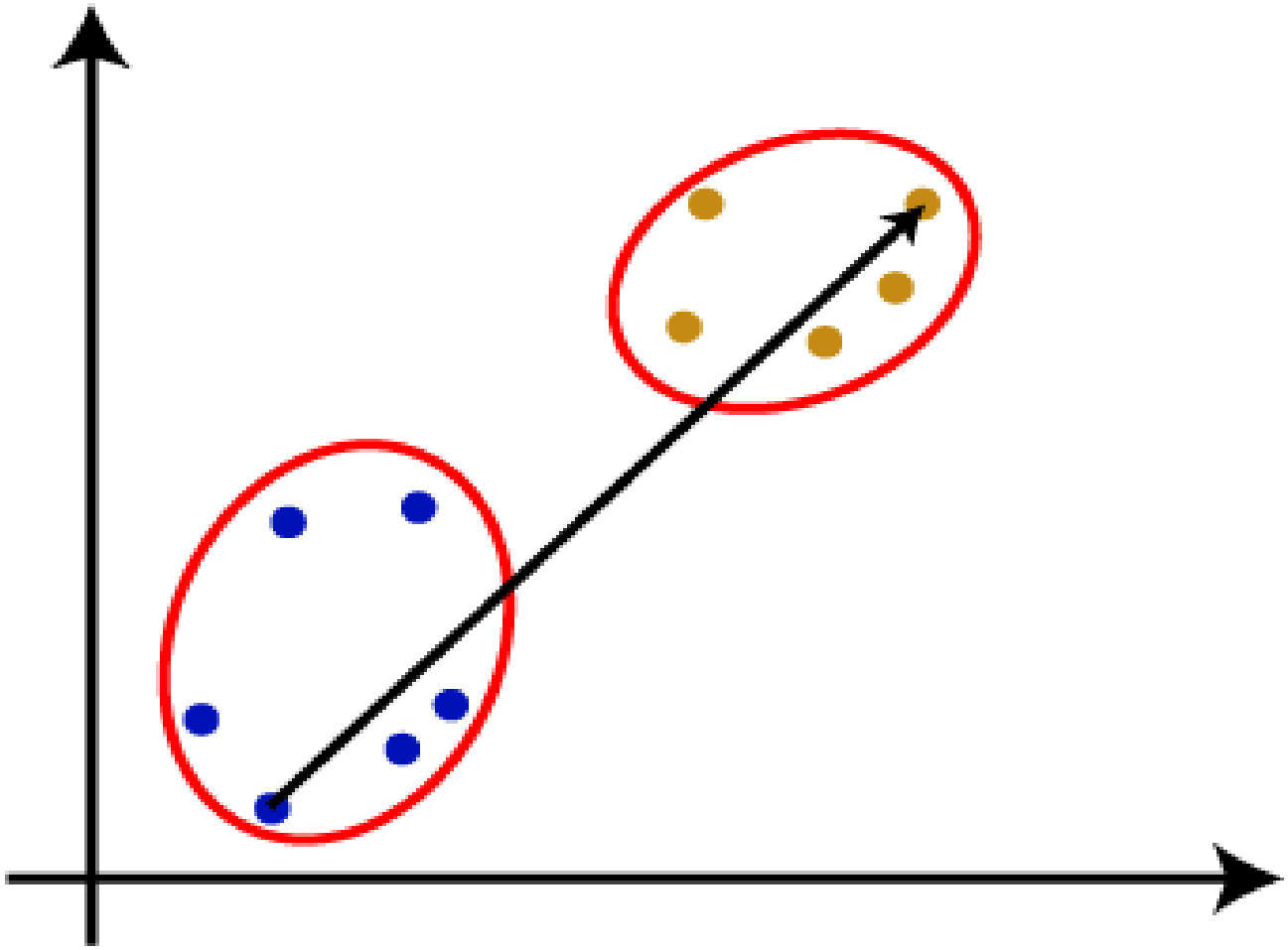
- These measures are called **Linkage methods**.

- Some of the popular linkage methods are given below:
  1. **Single Linkage**
  2. **Complete Linkage**
  3. **Average Linkage**
  4. **Centroid Linkage**

- **Single Linkage:** It is the Shortest Distance between the closest points of the clusters. Consider the below image:



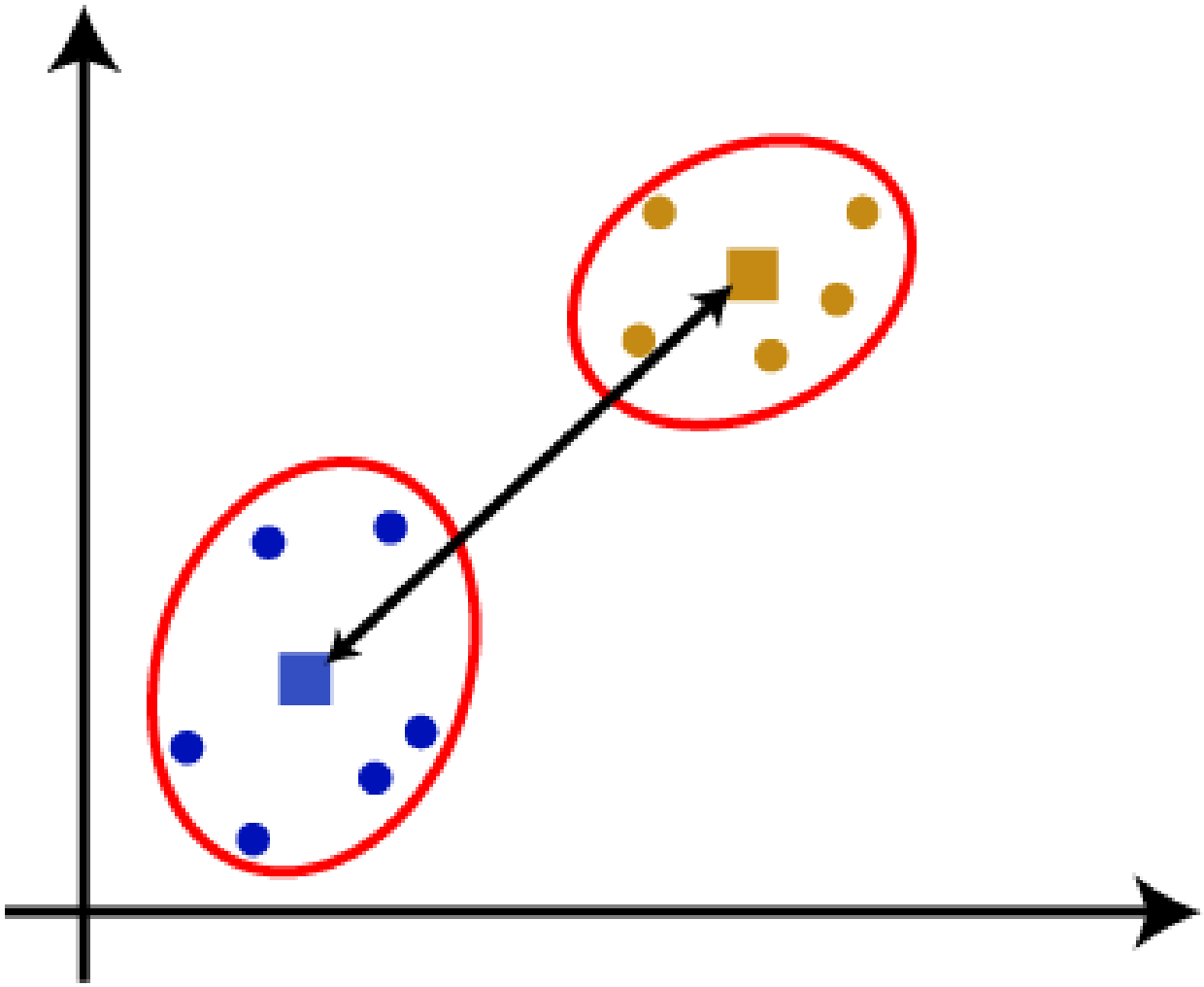
- **Complete Linkage:** It is the farthest distance between the two points of two different clusters. It is one of the popular linkage methods as it forms tighter clusters than single-linkage.



- **Average Linkage:** It is the linkage method in which the distance between each pair of datasets is added up and then divided by the total number of datasets to calculate the average distance between two clusters.

- It is also one of the most popular linkage methods.

- **Centroid Linkage:** It is the linkage method in which the distance between the centroid of the clusters is calculated. Consider the below image:



One of the core measures of proximities between clusters is the distance between them. There are four standard methods to measure the distance between clusters:

Let C_i and C_j be the two clusters with n_i and n_j respectively. p_i and p_j represents the points in clusters C_i and C_j respectively. We will denote the mean of cluster C_i as m_i.

Minimum distance $D_{\min}(C_i, C_j) = \min_{p_i \in C_i,\, p_j \in C_j} \{|p_i - p_j|\}$ (9.4)

Maximum distance $D_{\max}(C_i, C_j) = \max_{p_i \in C_i,\, p_j \in C_j} \{|p_i - p_j|\}$ (9.5)

Mean distance $D_{\text{mean}}(C_i, C_j) = \{|m_i - m_j|\}$ (9.6)

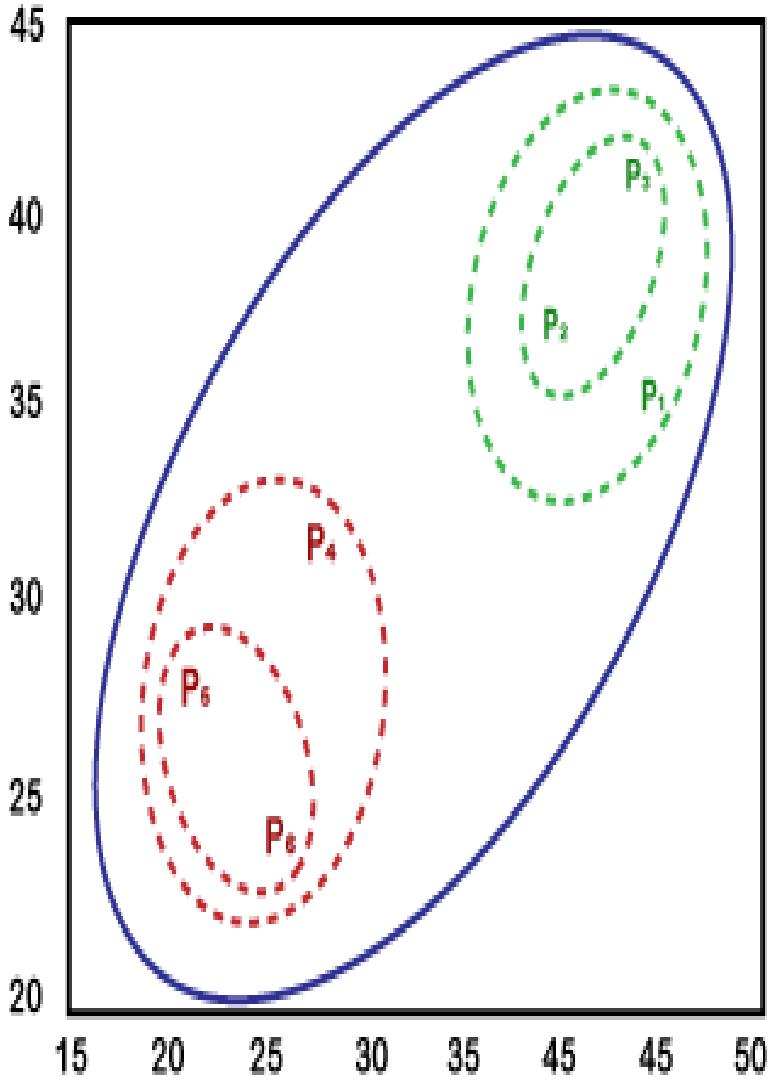
Average distance $D_{\text{avg}}(C_i, C_j) = \frac{1}{n_i n_j} \sum_{p_i \in C_i,\, p_j \in C_j} |pi - pj|$ (9.7)

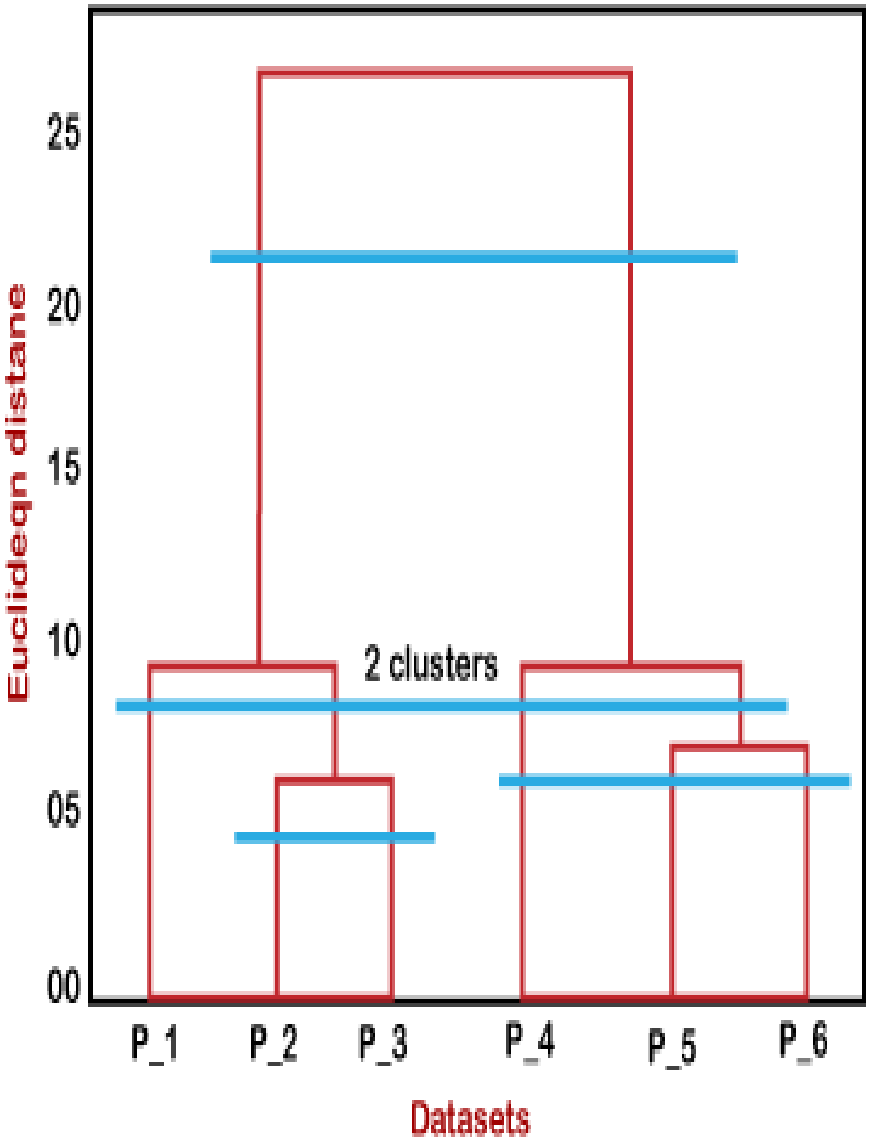
What is a Dendrogram?

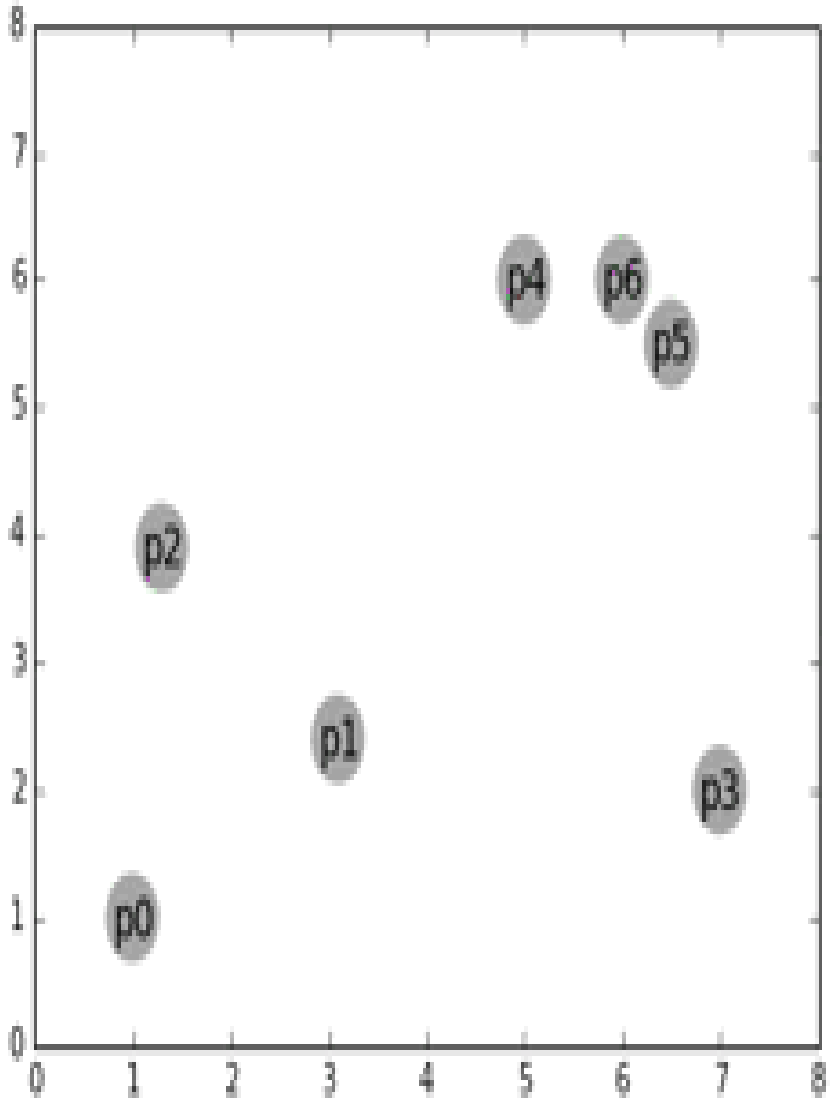
- A **Dendrogram is a type of tree diagram showing hierarchical relationships between different sets of data.**

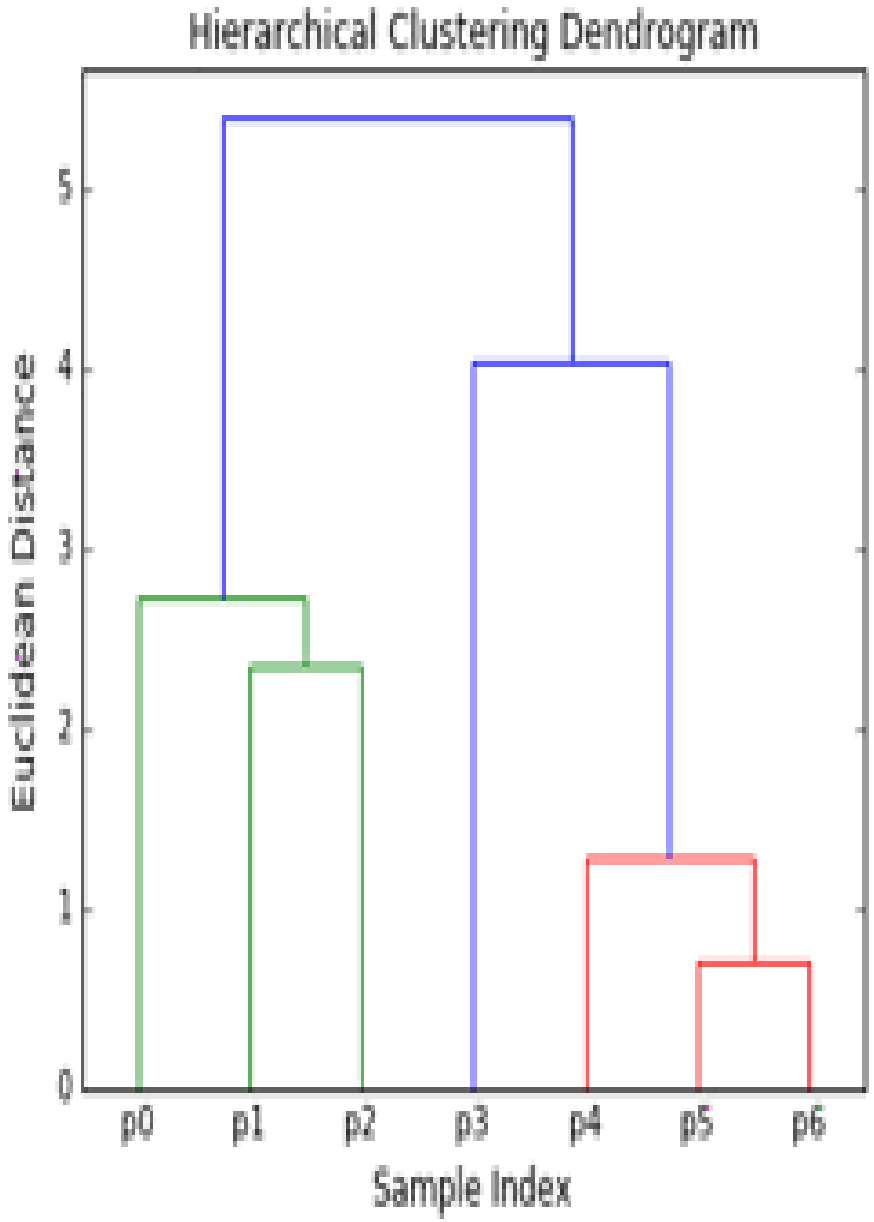
- The dendrogram is a tree-like structure that **is mainly used to store each step as a memory that the HC algorithm performs.**

- In the dendrogram plot, the **Y-axis shows the Euclidean distances between the data points**, and the **x-axis shows all the data points of the given dataset.**

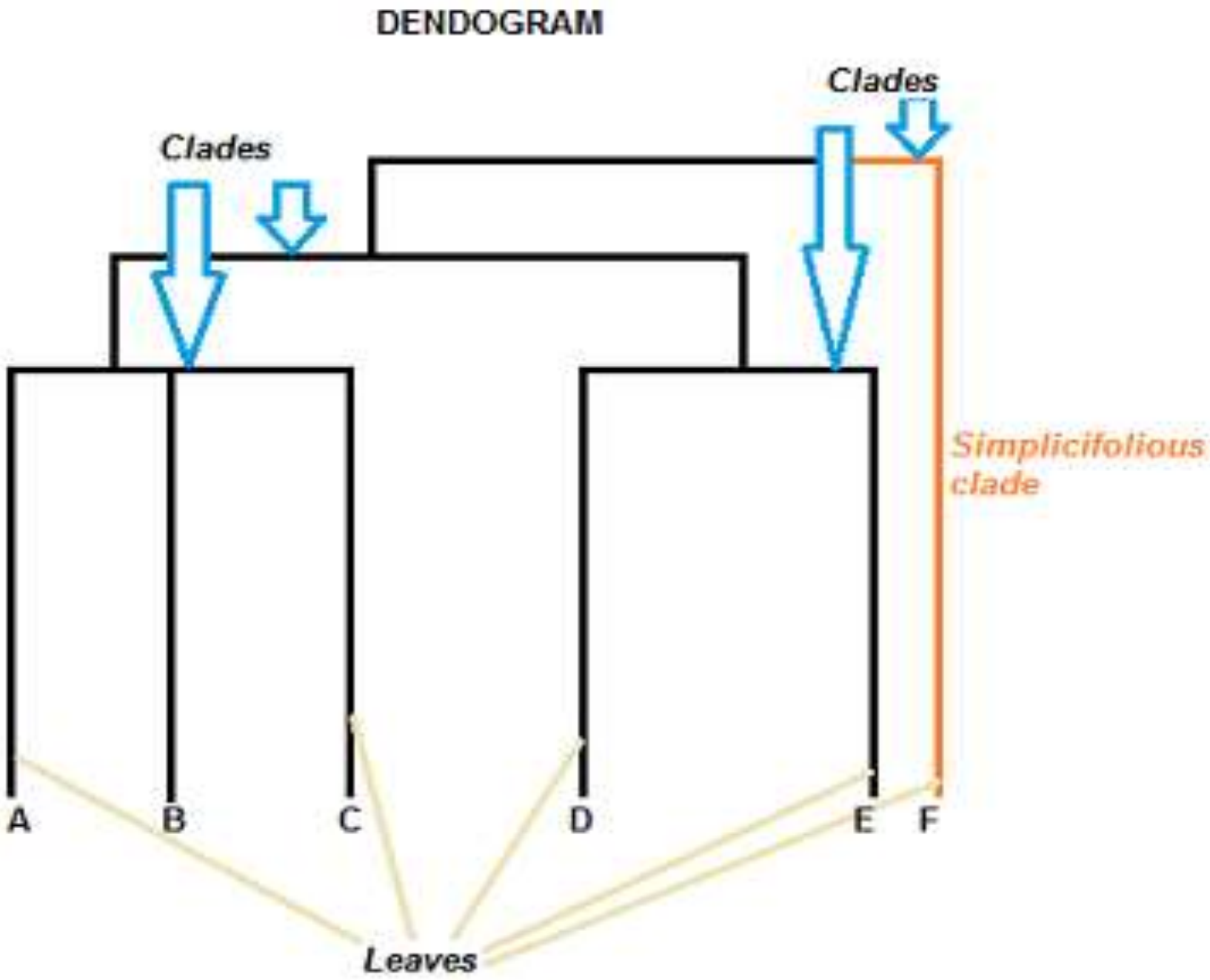

45
40
35
30
25
20
15
20
25
30
35
45
45
50
P3
P2
P1
P4
P5
P6


Euclideqn distane
25
20
15
10
05
00
2 clusters
P_1
P_2
P_3
P_4
P_5
P_6
Datasets


p0
p1
p2
p3
p4
p5
p6
0
1
2
3
4
5
6
7
8


Hierarchical Clustering Dendrogram
Euclidean Distance
Sample Index
p0
p1
p2
p3
p4
p5
p6
0
1
2
3
4
5

- Initially P5 and P6 which are closest to each other by any other point are combined into one cluster followed by P4 getting merged into the same cluster(C2).
- Then P1and P2 gets combined into one cluster followed by P0 getting merged into the same cluster(C4).
- Now P3 gets merged in cluster C2 and
- finally, both clusters get merged into one.

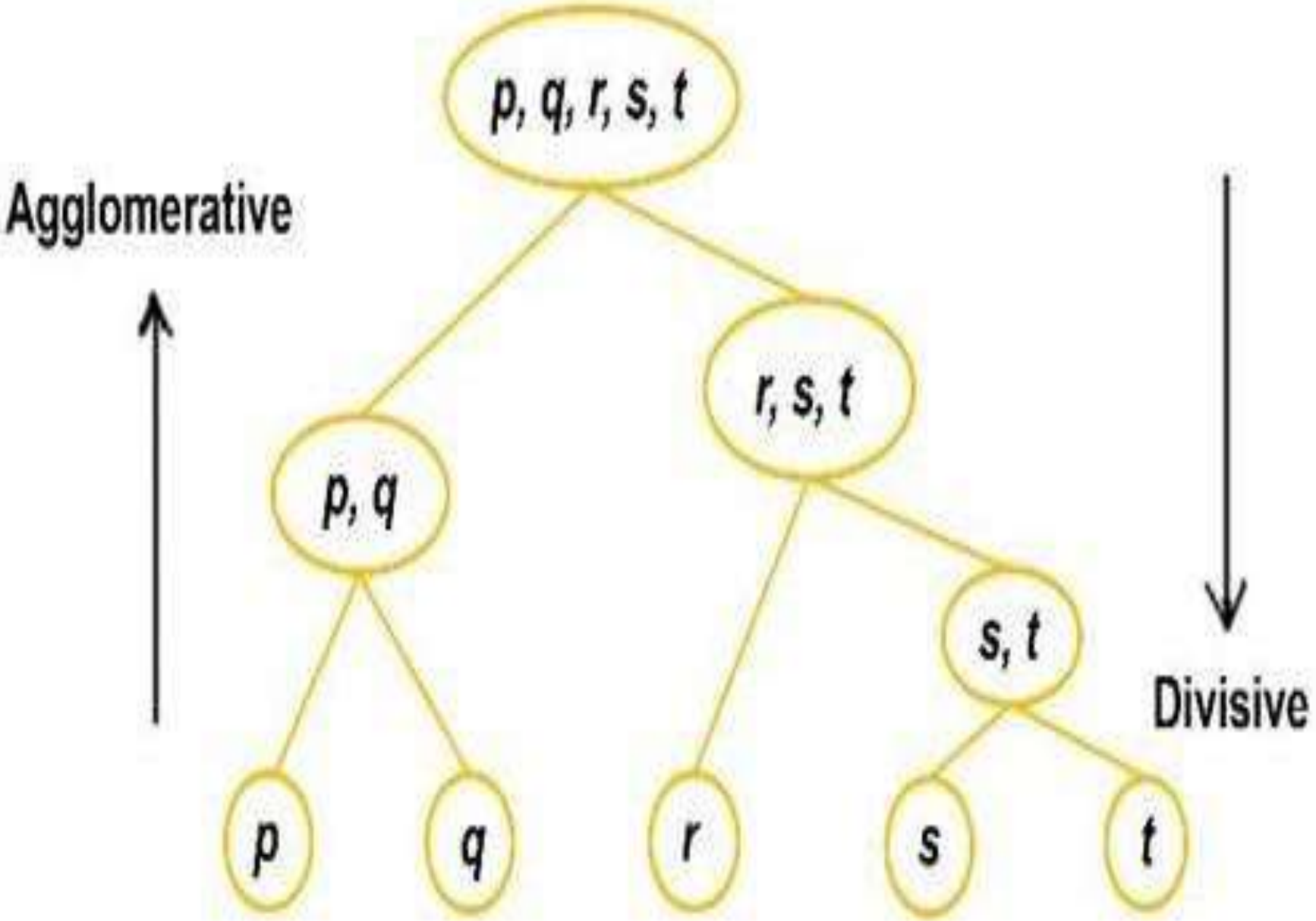

DENDOGRAM
Clades
Clades
Simplicifolious clade
A
B
C
D
E
F
Leaves

Divisive Hierarchical Clustering

- In *Divisive* or **DIANA(DIvisive ANAlysis Clustering**) is a **top-down clustering** method where we assign **all of the observations to a single cluster and then partition the cluster to two least similar clusters.**

- Finally, we **proceed recursively on each cluster until there is one cluster for each observation.**

- So this clustering approach is exactly **opposite to Agglomerative clustering.**


p, q, r, s, t
Agglomerative
r, s, t
p, q
s, t
Divisive
p
q
r
s
t

Density-Based Clustering

- The density-based clustering method connects the **highly-dense** (Dark, Thick) areas into clusters, and the randomly shaped distributions are formed as long as the dense region can be connected.

- This algorithm does it by **identifying different clusters in the dataset and connects the areas of high densities into clusters.**

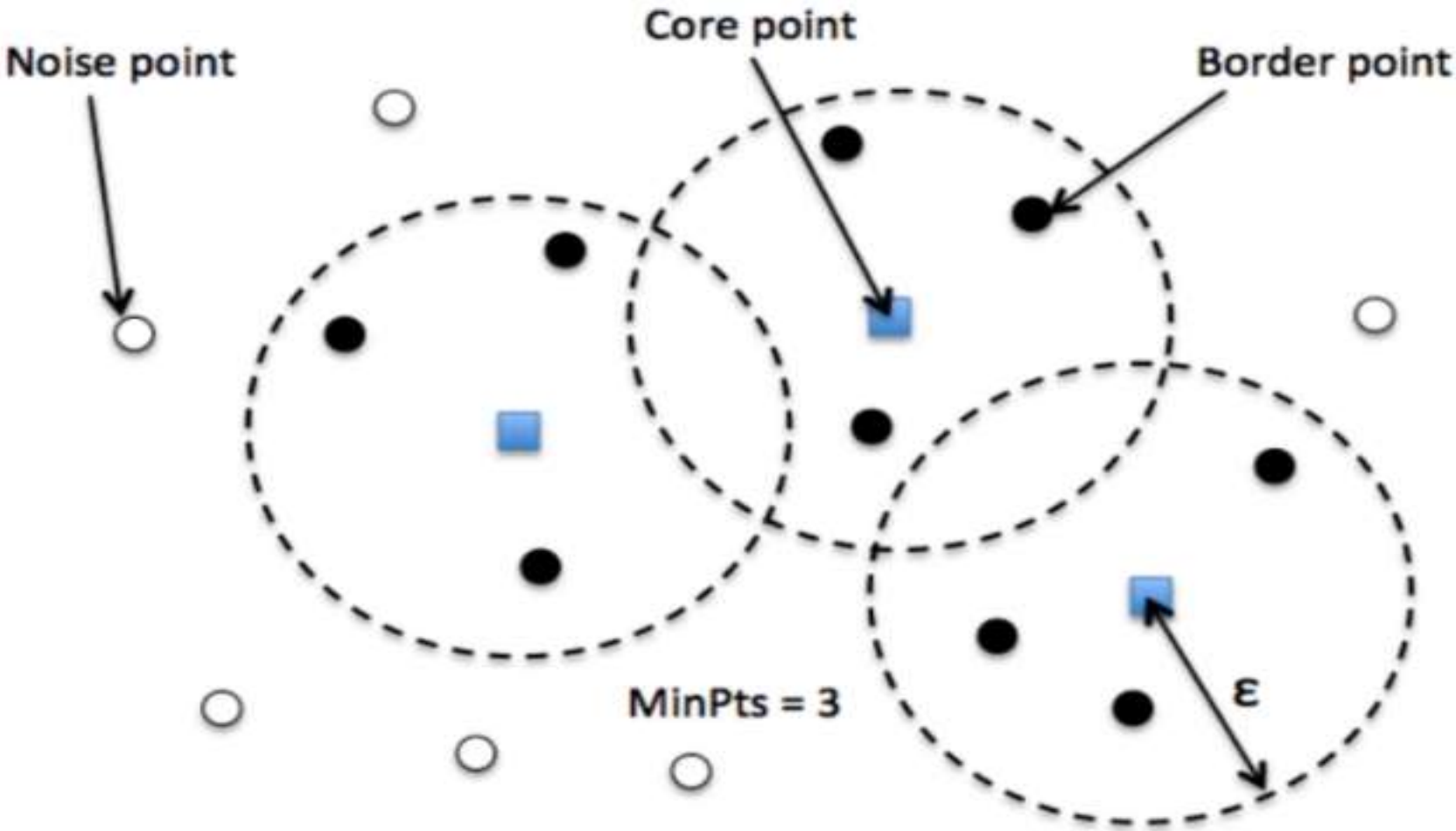
- The **dense areas in data space are divided from each other by sparser areas.**

- These algorithms can face difficulty in clustering the data points if the dataset has varying densities and high dimensions.

- **Density-Based Spatial Clustering of Applications with Noise (DBSCAN)** is a base algorithm for density-based clustering.

- It can discover clusters of different shapes and sizes from a large amount of data, which is containing noise and outliers.

- The DBSCAN algorithm uses two parameters:
  - **minPts:** The minimum number of points (a threshold) clustered together for a region to be considered dense.
  - **eps (ε):** A distance measure that will be used to locate the points in the neighborhood of any point.



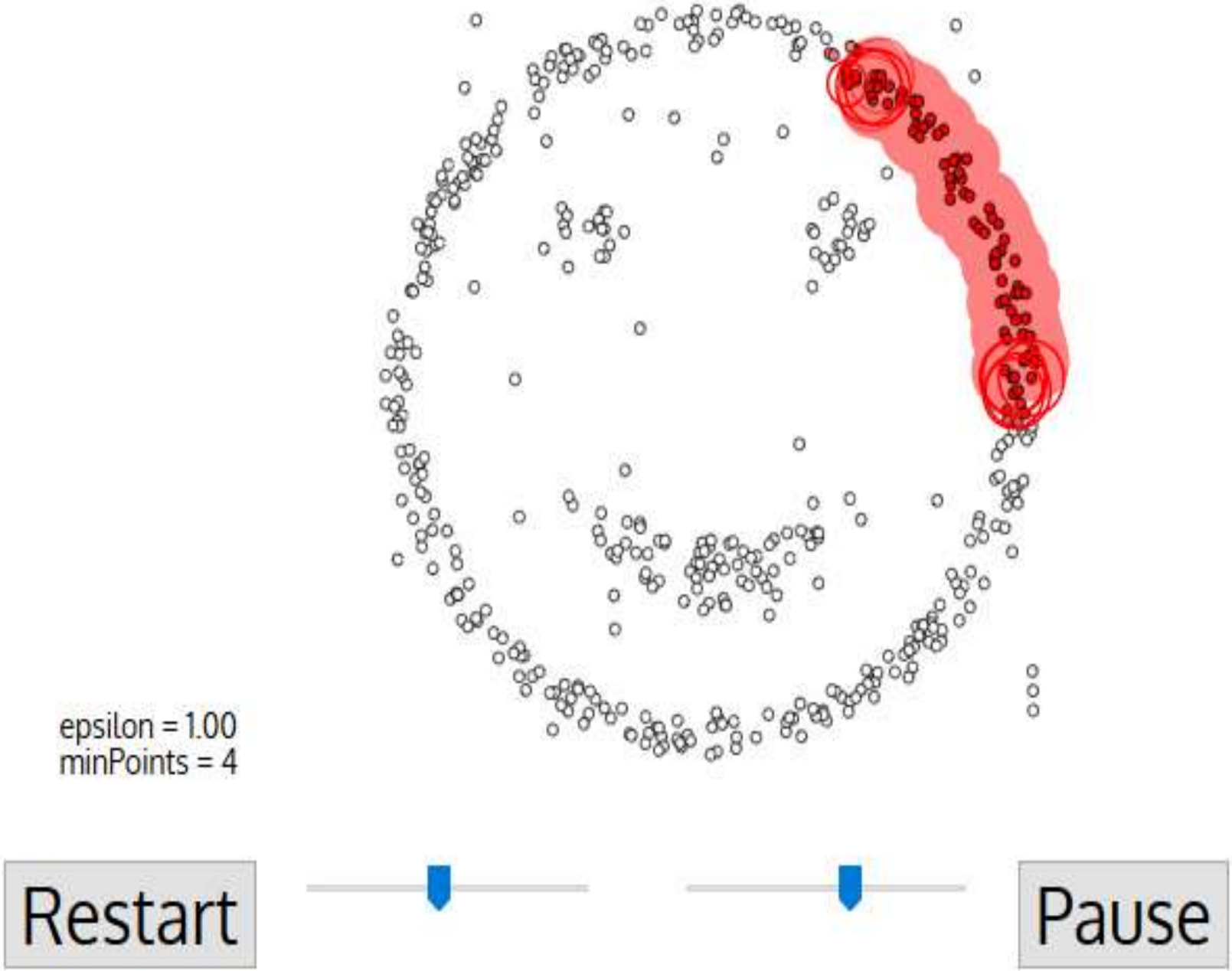
- **Core** — This is a point that has at least *m* points within distance *n* from itself.
- **Border** — This is a point that has at least one Core point at a distance *n*.
- **Noise** — This is a point that is neither a Core nor a Border. And it has less than *m* points within distance *n* from itself.

Algorithmic steps for DBSCAN clustering

- The algorithm proceeds by **arbitrarily picking up a point in the dataset (until all points have been visited).**

- If there **are at least 'minPoint' points within a radius of 'ε' to the point then we consider all these points to be part of the same cluster**.

- The clusters are then **expanded by recursively repeating the neighborhood calculation for each neighboring point**


epsilon = 1.00
minPoints = 4
Restart
Pause

Association Rule Learning

- **Association rule** learning is a type of unsupervised learning technique that checks for the **dependency of one data item on another data item and maps accordingly so that it can be more profitable.**

- It **tries to find some interesting relations or associations among the variables of dataset.**

- It is **based on different rules to discover the interesting relations between variables in the database.**

- The association rule learning is one of the very important concepts of machine learning, and it is employed in **Market Basket analysis, Web usage mining, continuous production, etc**

Market Basket Analysis

- Market basket analysis is a **technique used by the various big retailer to discover the associations between items.**

- They try to **find out associations between different items** and products that **can be sold together**, which gives assisting in right product placement.

- Typically, it figures out what products are being bought together and organizations can place products in a similar manner.

- **Example:** People who buy Bread usually buy Butter too. The Marketing teams at retail stores should target customers who buy bread and butter and provide an offer to them so that they buy the third item, like eggs.

- So if **customers buy bread and butter and see a discount or an offer on eggs, they will be encouraged to spend more and buy the eggs**.
- This is what market basket analysis is all about.

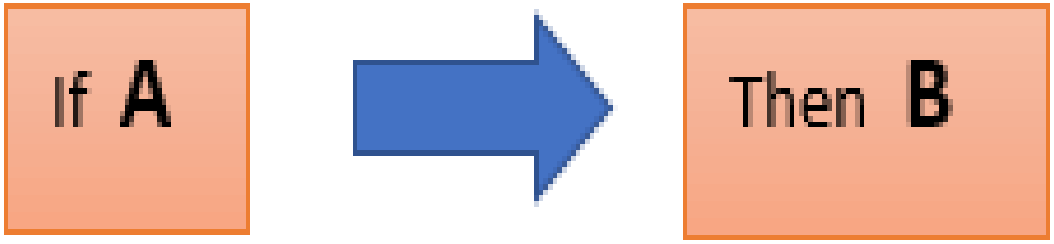
- **For example,** if a customer buys bread, he most likely can also buy butter, eggs, or milk, so these products are stored within a shelf or mostly nearby.

- Association rule learning can be divided into three types of algorithms:

  - **Apriori**
  - **Eclat**
  - **F-P Growth Algorithm**

How does Association Rule Learning work?

- Association rule learning works on the concept of If and Else Statement, such as if A then B.



- Here the If element is called **antecedent**, and
- Then statement is called as **Consequent**.
- These types of relationships where we can find out some association or **relation between two items is known *as single cardinality.***
- It is all about creating rules, and **if the number of items increases, then cardinality also increases accordingly.**

- To measure the associations between thousands of data items, there are several metrics.
- These metrics are given below:
  - **Support**
  - **Confidence**
  - **Lift**

Support

- Support is the frequency of A or **how frequently an item appears in the dataset.**

- It is defined as the **fraction of the transaction T that contains the itemset X.**

- If there are X datasets, then for transactions T, it can be written as:

$$\text{Supp}(X) = \frac{Freq(X)}{T}$$

Confidence

- Confidence indicates how often the rule has been found to be true.

- Or **how often the items X and Y occur together in the dataset when the occurrence of X is already given.**

- It is the ratio of the transaction that contains X and Y to the number of records that contain X.

$$\text{Confidence} = \frac{Freq(X,Y)}{Freq(X)}$$

Lift

- It is the strength of any rule, which can be defined as below formula:

$$\text{Lift} = \frac{Supp(X,Y)}{Supp(X) \times Supp(Y)}$$

- It is the ratio of the observed support measure and expected support if X and Y are independent of each other.
- It has three possible values:
  - If **Lift= 1**: The probability of **occurrence of antecedent and consequent is independent of each other.**
  - **Lift>1**: It determines the degree to **which the two itemsets are dependent to each other.**
  - **Lift<1**: It tells us that **one item is a substitute for other items, which means one item has a negative effect on another.**

Apriori Algorithm

- The Apriori algorithm **uses frequent itemsets to generate association rules,** and **it is designed to work on the databases that contain transactions.**

- With the help of these association rule, **it determines how strongly or how weakly two objects are connected.**

- This algorithm uses a **breadth-first search** and **Hash Tree** to calculate the itemset associations efficiently.

- It is the **iterative process for finding the frequent itemsets from the large dataset.**

- This algorithm was given by the **R. Agrawal** and **Srikant** in the year **1994**.

- It is **mainly used for *market basket analysis* and helps to find those products that can be bought together.**

- It can also be used in the healthcare field to find drug reactions for patients.

What is Frequent Itemset?

- Frequent itemsets are **those items whose support is greater than the threshold value or user-specified minimum support.**

- It means if A & B are the frequent itemsets together, then individually A and B should also be the frequent itemset.

- Suppose there are the two transactions: **A= {1,2,3,4,5},** and **B= {2,3,7},** in these two transactions, **2 and 3 are the frequent itemsets**.

Steps for Apriori Algorithm

- Below are the steps for the apriori algorithm:
- **Step-1:** Determine the support of itemsets in the transactional database, and **select the minimum support and confidence.**
- **Step-2:** Take all supports in the transaction with **higher support value than the minimum or selected support value.**
- **Step-3:** Find all the rules of these subsets that **have higher confidence value than the threshold or minimum confidence.**
- **Step-4:** Sort the rules as the decreasing order of lift.

Apriori Algorithm Working

- **Example:** Suppose we have the following dataset that has various transactions, and from this dataset, we need to **find the frequent itemsets and generate the association rules using the Apriori algorithm:**

TID	ITEMSETS
T1	A, B
T2	B, D
T3	B, C
T4	A, B, D
T5	A, C
T6	B, C
T7	A, C
T8	A, B, C, E
T9	A, B, C

Given: Minimum Support= 2, Minimum Confidence= 50%

Solution:

Step-1: Calculating C1 and L1:

- In the first step, we will create a table that contains support count (The frequency of each itemset individually in the dataset) of each itemset in the given dataset. This table is called the **Candidate set or C1.**

itemset	Support_count
A	6
B	7
C	6
D	2
E	1

- Now, we will take out all the itemsets that have the greater support count that the Minimum Support (2). It will give us the table for the **frequent itemset L1.**
  Since all the itemsets have greater or equal support count than the minimum support, except the E, so E itemset will be removed.

itemset	Support_count
A	6
B	7
C	6
D	2

Step-2: Candidate Generation C2, and L2:

- In this step, we will generate C2 with the help of L1. In C2, we will create the pair of the itemsets of L1 in the form of subsets.
- After creating the subsets, we will again find the support count from the main transaction table of datasets, i.e., how many times these pairs have occurred together in the given dataset. So, we will get the below table for C2:

Itemset	Support_Count
{A, B}	4
{A,C}	4
{A, D}	1
{B, C}	4
{B, D}	2
{C, D}	0

- Again, we need to compare the C2 Support count with the minimum support count, and after comparing, the itemset with less support count will be eliminated from the table C2. It will give us the below table for L2

Itemset	Support_Count
{A, B}	4
{A, C}	4
{B, C}	4
{B, D}	2

A, B, C, D

Step-3: Candidate generation C3, and L3:

- For C3, we will repeat the same two processes, but now we will form the C3 table with subsets of three itemsets together, and will calculate the support count from the dataset. It will give the below table:

itemset	Support_count
{A,B,C}	2
{B,C,D}	0
{A,C,D}	0
{A,D,C}	0
{A,B,D}	1

- Now we will create the L3 table. As we can see from the above C3 table, there is only one combination of itemset that has support count equal to the minimum support count. So, the L3 will have only one combination, i.e., **{A, B, C}**.

Step-4: Finding the association rules for the subsets:

To generate the association rules, first, we will create a new table with the possible rules from the occurred combination {A, B.C}. For all the rules, we will calculate the Confidence using formula **sup(A ^B)/A.** After calculating the confidence value for all rules, we will exclude the rules that have less confidence than the minimum threshold(50%).

Consider the below table:

A ^B → C	**2**	**Sup{(A ^B) ^C}/sup(A ^B)= 2/4=0.5=50%**
B^C → A	2	Sup{(B^C) ^A}/sup(B ^C)= 2/4=0.5=50%
A^C → B	2	Sup{(A ^C) ^B}/sup(A ^C)= 2/4=0.5=50%
C→ A ^B	2	Sup{(C^(A ^B)}/sup(C)= 2/6=0.33=33.33%
A→ B^C	2	Sup{(A^(B ^C)}/sup(A)= 2/6=0.33=33.33%
B→ B^C	2	Sup{(B^(B ^C)}/sup(B)= 2/7=0.28=28%

- As the given threshold or minimum confidence is 50%, so the first three rules **A ^B → C, B^C → A, and A^C → B** can be considered as the strong association rules for the given problem.

- https://www.kdnuggets.com/
- Edureka
- Javatpoint